Table of Content

Module 1:

1 Learning goals for the course

In this course you will be introduced to a Data Scientist's workbench or toolkit that consists of a variety of tools, languages, libraries, APIs, data sets, models, etc. used by Data Scientists. Try not to be overwhelmed by the sheer number of components and tools that exist in the Data Science ecosystem. The main goal of the course is for you to be knowledgeable about the kinds of tools Data Scientists use, their examples, and get some hands-on time with a few key tools.

As such, you are not required to recall the name of every single tool covered in the course. However, be familiar with the categories or types of tools and 1 or 2 examples each type. Modules 4 and 5 of this course will cover some of the most important ones for a beginner Data Scientist in greater depth and enable you to get hands-on experience with them. As you take additional Data Science courses, you will become more acquainted with some of the other tools and libraries. Some may be required to perform more specialized or advanced Data Science or Machine Learning tasks. So don't try to remember all of the names just now. Pay special attention, though, to Video and Lesson summaries.

To successfully complete the course, you are required to complete the first 6 out of the 7 modules in the course. The 7th module is an optional one.

Here is what you will be learning in each module:

Module 1: Overview of Data Science Tools

In this module, you will learn about the different types and categories of tools that data scientists use and popular examples of each. You will also become familiar with Open Source, Cloud-based, and Commercial options for data science tools.

- Describe the components of a Data Scientist's toolkit and list various tool categories
- List examples of Open Source, Commercial, and Cloud-based tools in various categories

Module 2: Languages of Data Science

This module will bring awareness about the criteria determining which language you should learn. You will learn the benefits of Python, R, SQL, and other common languages such as Java, Scala, C++, JavaScript, and Julia. You will explore how you can use these languages in Data Science. You will also look at some sites for more information about the languages.

1. Learning Objectives

- Identify the criteria and roles for determining the language to learn.
- Identify the users and benefits of Python.
- Identify the users and uses of the R language.
- Define SQL elements and list their benefits.
- Review languages such as Java, Scala, C++, JavaScript, and Julia.

- List the global communities for connecting with other users.

Module 3: Packages, APIs, Data Sets and Models

This module will give you in-depth knowledge of different libraries, APIs, dataset sources and models used by data scientist.

Learning Objectives

- List examples of the various libraries: scientific, visualization, machine learning, and deep learning.
- Define REST API to request and respond.
- Describe data sets and sources of data.
- Explore open data sets on the Data Asset eXchange.
- Describe how to use a learning model to solve a problem.
- List the tasks that a data scientist needs to perform to build a model.
- Explore ML models in the Model Learning eXchange.

Module 4: Jupyter Notebooks and JupyterLab

This module introduces the Jupyter Notebook and JupyterLab. You will learn how to work with different kernels and the basic Jupyter architecture. In addition, you will identify the tools in an Anaconda Jupyter environment. Finally, the module overviews cloud-based Jupyter environments and their data science features.

Learning Objectives

- Describe how to use the notebooks in JupyterLab.
- Describe how to work in a notebook session.
- Describe the basic Jupyter architecture.
- Describe how to work with kernels.
- Identify tools in Anaconda Jupyter environments.
- Describe cloud-based Jupyter environments and their data science features.

Module 5: RStudio and GitHub

This module will start with an introduction to R and RStudio and will end up with Github usage. You will learn about the different R visualization packages and how to create visual charts using the plot function.

Further in the module, you will develop the essential conceptual and hands-on skills to work with Git and GitHub. You will start with an overview of Git and GitHub, creating a GitHub account and a project repository, adding files, and committing your changes using the web interface. Next, you will become familiar with Git workflows involving branches, pull requests (PRs), and merges. You will also complete a project at the end to apply and demonstrate your newly acquired skills.

Learning Objectives

- Describe R capabilities and RStudio environment.
- Use the inbuilt R plot function.
- Explain version control and describe the Git and GitHub environment.

- Describe the purpose of source repositories and explain how GitHub satisfies the needs of a source repository.
- Create a GitHub account and a project repository.
- Demonstrate how to edit and upload files in GitHub.
- Explain the purpose of branches and how to merge changes.

Module 6: Final Project and Assessment

In this module, you will work on a final project to demonstrate some of the skills learned in the course. You will also be tested on your knowledge of various components and tools in a Data Scientist's toolkit learned in the previous modules.

Learning Objectives

- Create a Jupyter Notebook with markdown and code cells
- List examples of languages, libraries and tools used in Data Science
- Share your Jupyter Notebook publicly on GitHub
- Evaluate notebooks submitted by your peers using the provided rubric
- Demonstrate proficiency in Data Science toolkit knowledge

Module7: IBM Watson Studio

This is as an optional module if you are interested in learning about and working with data science tools from IBM such as Watson Studio.

2. Learning Objectives

- Find common resources in Watson Studio and IBM Cloud Pak for Data.
- Create an IBM Cloud account, service, and project in Watson Studio.
- Create and share a Jupyter Notebook.
- Use different types of Jupyter Notebook templates and kernels on IBM Watson Studio.
- Describe how to connect a Watson Studio account and publish a notebook in GitHub.

2 Data Science Task Categories

Before raw data can be useful, it must pass through various data science task categories. These categories include:

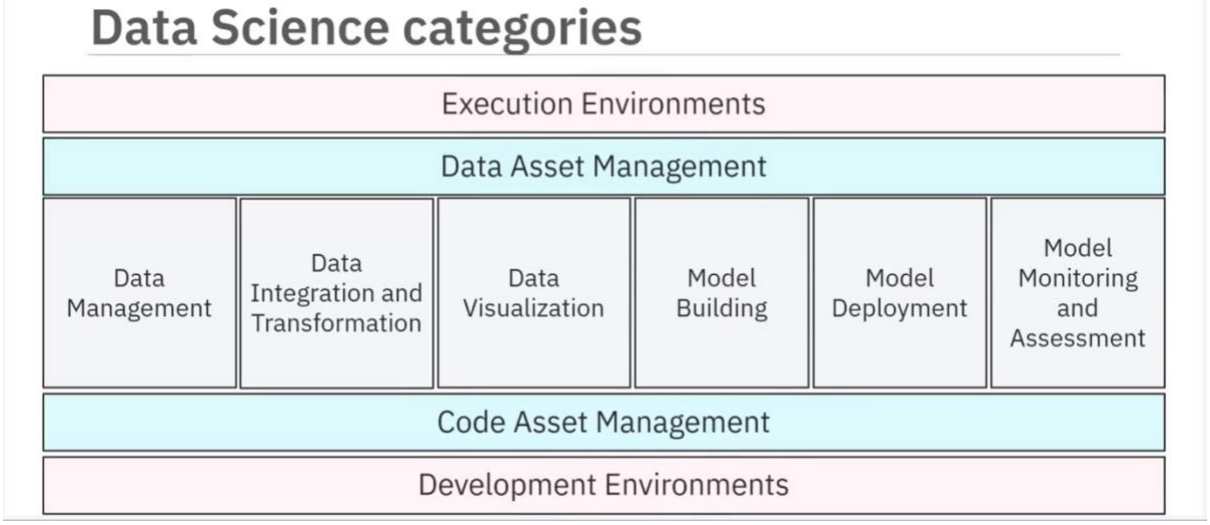
- Data Management
- Data Integration and Transformation

- Data Visualization
- Model Building
- Model Deployment
- Model Monitoring and Assessment

To perform these tasks effectively, data scientists use the following tools:

- Data Asset Management
- Code Asset Management
- Execution Environments
- Development Environments

Let's explore each category in detail.



2.1 Data Management

Data management involves collecting, storing, and retrieving data securely, efficiently, and cost-effectively. Data can be collected from various sources, such as:

- Social Media (e.g., Twitter, Flipkart)
- Media Platforms
- Sensors

Once collected, the data is stored in persistent storage, ensuring availability when needed.

2.2 Data Integration and Transformation

Data Integration and Transformation involves the process of **Extracting, Transforming, and Loading (ETL)** data. This process includes:

- Extracting data from multiple repositories (e.g., databases, data cubes, flat files)
- Storing extracted data in a central repository like a **Data Warehouse**
- Transforming data by modifying values, structure, and format (e.g., converting height and weight to metric units)

- Loading transformed data back into the Data Warehouse for analysis

2.3 Data Visualization

Data visualization is the graphical representation of data and information. It helps decisionmakers interpret data more effectively. Various types of visualizations include:

- **Bar Charts** – Compare the size of each component
- **Treemaps** – Display hierarchical data
- **Line Charts** – Plot data points over time
- **Map Charts** – Display data by location (e.g., geographical maps, website usage maps)

2.4 Model Building

Model building involves training data and analyzing patterns using **Machine Learning algorithms**. The system learns how to make predictions or decisions based on data. This process can be performed using tools like **IBM Watson Machine Learning**, which provides a comprehensive set of services for building models.

2.5 Model Deployment

Model deployment integrates a developed model into a production environment. This process involves:

- Making the model available to third-party applications via **APIs**
- Enabling business users to interact with and analyze data
- Using tools like **SPSS Collaboration and Deployment Services** to deploy assets created with SPSS software

2.6 Model Monitoring and Assessment

Model monitoring and assessment ensure the model's accuracy, fairness, and robustness. This involves:

- **Model Monitoring** – Tools like **Fiddler** track the performance of deployed models in production environments.
- **Model Assessment** – Evaluation metrics like **F1 Score**, **True Positive Rate**, and **Sum of Squared Error** measure performance.
- **IBM Watson OpenScale** – Continuously monitors deployed machine learning and deep learning models, improving prediction accuracy.

2.7 Supporting Tools for Data Science Tasks

2.7.1 Code Asset Management

Code asset management helps in organizing and managing code efficiently. Key features include:

- **Version Control** – Tracks changes to code, enabling updates, bug fixes, and feature improvements
- **Centralized Repositories** – Allow multiple developers to collaborate on the same project
- **Collaboration Platforms** – Examples include **GitHub**, which offers web-based version control, sharing, and access control

2.7.2 Data Asset Management

Data asset management, also known as **Digital Asset Management (DAM)**, organizes and manages important data collected from various sources. DAM platforms provide:

- Versioning and collaboration features
- Replication, backup, and access right management

2.7.3 Development Environments

Development Environments, also called **Integrated Development Environments (IDEs)**, provide a workspace for developing, testing, and deploying source code. IDEs such as **IBM Watson Studio** offer:

- Testing and simulation tools to emulate real-world scenarios
- A suite of tools for model development and execution

2.7.4 Execution Environments

An execution environment contains:

- Libraries for compiling source code
- System resources to execute and verify code
- Cloud-based execution environments, which provide flexibility and scalability

IBM Watson Studio is an example of a cloud-based execution environment that supports data preprocessing, model training, and deployment.

2.8 Conclusion

In this module, we covered the major **Data Science Task Categories**:

- Data Management
- Data Integration and Transformation
- Data Visualization
- Model Building
- Model Deployment
- Model Monitoring and Assessment

We also explored supporting tools, including:

- Data Asset Management

- Code Asset Management
- Execution Environments
- Development Environments

These tools enable data scientists to efficiently manage, process, visualize, and deploy datadriven models.

3 Open-Source Tools for Data Science Part 1

3.1 Introduction

Welcome to **Open-Source Tools for Data Science Part 1**. After completing this section, you will be able to:

- List the open-source data management tools.
- List the open-source data integration and transformation tools.
- List the data visualization tools.
- List the model tools for building, deployment, monitoring, and assessment.
- List tools for code and data asset management.

3.2 Open-Source Data Management Tools

The most widely used open-source data management tools include:

- **Relational Databases**: MySQL, PostgreSQL
- **NoSQL Databases**: MongoDB, Apache CouchDB, Apache Cassandra
- **File-Based Tools**: Hadoop File System, Cloud File Systems (Ceph)
- **Search Tools**: Elasticsearch (stores text data and create a search index for fast document retrieval)

3.3 Open-Source Data Integration and Transformation Tools

Data integration and transformation in traditional data warehousing follows the **Extract, Transform, Load (ETL)** approach, while data scientists often use **Extract, Load, Transform (ELT)**. This process is also known as **Data Refinery and Cleansing**. Common open-source tools include:

- **Apache AirFlow** – Created by Airbnb, used for workflow automation.
- **KubeFlow** – Runs data science pipelines on Kubernetes.
- **Apache Kafka** – Developed by LinkedIn, used for stream processing.
- **Apache Nifi** – Features a visual editor for data flow automation.
- **Apache SparkSQL** – Supports ANSI SQL and scales to thousands of compute nodes.
- **NodeRED** – Offers a lightweight, visual interface and runs on small devices like Raspberry Pi.

3.4 Open-Source Data Visualization Tools

Data visualization tools can be categorized into programming libraries and user-interfacebased tools. Common tools include:

- **Pixie Dust** – A Python-based library with a user interface for easy plotting.
- **Hue** – Creates visualizations from SQL queries.
- **Kibana** – A web application for data exploration and visualization, primarily for Elasticsearch.
- **Apache Superset** – A web application for data visualization and exploration.

3.5 Model Development

The missing information has now been included below.

IBM offers various tools and platforms tailored for model development across various domains. Here are a few examples:

- **IBM Watson Studio:** Engineered as an integrated environment, Watson Studio simplifies developing, training, and deploying models. It boasts support for multiple languages and frameworks, such as Python, R, and TensorFlow, alongside collaboration features, data preparation tools, and versatile deployment options.

- **IBM AutoAI:** A notable feature embedded within Watson Studio, IBM AutoAI streamlines the machine learning model construction process. By dynamically exploring various algorithms and hyperparameters, it aims to identify the optimal model for a given dataset.
- **IBM Watson OpenScale:** As a platform for overseeing and managing AI models in production, Watson OpenScale plays a pivotal role in ensuring model fairness, explainability, and bias mitigation. It furnishes insights into model performance and drift over time, facilitating informed decision-making.
- **IBM Watson Machine Learning:** Watson Machine Learning, available as a service on the IBM Cloud platform, enables users to scale their training and deployment of machine learning models. It seamlessly supports popular frameworks like TensorFlow, PyTorch, and sci-kit-learn, and offers APIs for seamless integration with other applications.

3.6 Open-Source Model Deployment Tools

Deploying machine learning models allows them to be consumed by developers via APIs. Key tools include:

- **Apache PredictionIO** – Supports Apache Spark ML models for deployment.
- **Seldon** – Supports multiple frameworks like TensorFlow, Apache SparkML, R, and scikit-learn. It runs on Kubernetes and Red Hat OpenShift.
- **MLeap** – Another way to deploy Apache SparkML models.
- **TensorFlow Serving** – Deploys TensorFlow models.
- **TensorFlow Lite** – Optimized for embedded devices like Raspberry Pi and smartphones.
- **TensorFlow.js** – Allows TensorFlow models to run in web browsers.

3.7 Open-Source Model Monitoring Tools

Monitoring deployed models ensures their performance remains optimal. Key tools include:

- **ModelDB** – A metadata repository for models, supporting Apache Spark ML Pipelines and scikit-learn.
- **Prometheus** – A widely used, multi-purpose monitoring tool.
- **IBM AI Fairness 360** – Detects and mitigates bias in machine learning models.
- **IBM Adversarial Robustness 360 Toolbox** – Identifies vulnerabilities to adversarial attacks and enhances model robustness.
- **IBM AI Explainability 360** – Helps interpret and explain machine learning model decisions.

3.8 Open-Source Code Asset Management Tools

Version control is essential for managing code efficiently. The most widely used tools include:

- **Git** – The de facto standard for version control.
- **GitHub** – The most popular Git-based repository hosting service.
- **GitLab** – An open-source alternative to GitHub, allowing self-hosting.
- **Bitbucket** – Another Git-based repository service.

3.9 Open-Source Data Asset Management Tools

Data asset management, also known as **data governance or data lineage**, ensures data is properly versioned and annotated with metadata. Key tools include:

- **Apache Atlas** – Supports enterprise-grade data governance.
- **ODPi Egeria** – An open ecosystem for metadata management, managed by the Linux Foundation.
- **Kylo** – A data management software platform supporting extensive data asset management tasks.

3.10 Summary

In this section, you learned about various open-source tools for data science, including:

- **Data Management Tools**: MySQL, PostgreSQL, MongoDB, Apache CouchDB, Apache Cassandra, Hadoop File System, Ceph, Elasticsearch.
- **Data Integration & Transformation Tools**: Apache AirFlow, KubeFlow, Apache Kafka, Apache Nifi, Apache SparkSQL, NodeRED.
- **Data Visualization Tools**: Pixie Dust, Hue, Kibana, Apache Superset.
- **Model Deployment Tools**: Apache PredictionIO, Seldon, Kubernetes, Redhat OpenShift, MLeap, TensorFlow Serving, TensorFlow Lite, TensorFlow.js.
- **Model Monitoring Tools**: ModelDB, Prometheus, IBM AI Fairness 360, IBM Adversarial Robustness 360 Toolbox, IBM AI Explainability 360.
- **Code Asset Management Tools**: Git, GitHub, GitLab, Bitbucket.
- **Data Asset Management Tools**: Apache Atlas, ODPi Egeria, Kylo.

4 Open-Source Tools for Data Science Part 2

4.1 Introduction

Welcome to *Open-Source Tools for Data Science Part 2.* After watching this video, you will be able to:

- Compare and contrast different open-source tools.
- Describe the relevant features of open-source tools.

4.2 Jupyter: A Popular Development Environment

Jupyter is one of the most widely used development environments for data scientists.
Originally designed for interactive Python programming, it now supports over a hundred different programming languages through "kernels," which encapsulate execution environments for different languages.

4.2.1 Key Features of Jupyter Notebooks

- Unifies documentation, code, output, shell commands, and visualizations into a single document.
- Jupyter Lab is the next version of Jupyter Notebooks, offering a more modern and modular structure.
- Allows opening and arranging different file types, including Jupyter Notebooks, data, and terminals, on a single canvas.

4.3 Apache Zeppelin: An Alternative to Jupyter

Apache Zeppelin, inspired by Jupyter Notebooks, provides a similar experience with additional features.

4.3.1 Key Features of Apache Zeppelin

- Integrated plotting capability without requiring external libraries.
- Extensible with additional libraries for enhanced functionality.

4.4 RStudio: A Development Environment for R

RStudio is one of the oldest development environments for statistics and data science, originating in 2011.

4.4.1 Key Features of RStudio

- Exclusively runs R and associated R libraries.
- Allows Python development within the R environment.
- Provides an optimal user experience with integration into Jupyter tools.
- Unifies programming, execution, debugging, remote data access, data exploration, and visualization.

4.5 Spyder: A Python-Based Alternative to RStudio

Spyder attempts to bring RStudio's functionality to Python. While it is not as feature-rich as RStudio, it is still considered a viable alternative.

4.6 Cluster Execution Environments

For handling large-scale data that exceeds a single computer's storage or memory, cluster execution environments are used.

4.6.1 Apache Spark: A Scalable Data Processing Engine

- One of the most active Apache projects, used by various industries, including Fortune 500 companies.

- Key property: **Linear scalability** – doubling the number of servers approximately doubles performance.

4.6.2 Apache Flink: Real-Time Stream Processing •

Developed as an alternative to Apache Spark.

- Unlike Apache Spark, which processes data in batches, Apache Flink focuses on real-time data streams.
- While both engines support batch and stream processing, Apache Spark remains the preferred choice for most use cases.

4.6.3 Ray: Optimized for Deep Learning

- One of the latest developments in data science execution environments.
- Primarily focused on large-scale deep learning model training.

4.7 Visual Open-Source Tools for Data Science

These tools offer fully integrated visual environments that require no programming knowledge. They support essential data science tasks, including:

- Data integration and transformation.
- Data visualization.
- Model building.

4.7.1 KNIME: A Visual Data Science Tool

- Originated from the University of Konstanz in 2004.
- Provides a visual user interface with drag-and-drop capabilities.
- Includes built-in visualization features.
- Extensible through R and Python programming.
- Supports connectors to Apache Spark.

4.7.2 Orange: A User-Friendly Alternative

- Easier to use but less flexible than KNIME.
- Provides essential data science functionalities.

4.8 Conclusion

In this video, you have learned about various open-source tools relevant to data science, including Jupyter, Apache Zeppelin, RStudio, Spyder, Apache Spark, Apache Flink, Ray, KNIME, and Orange.

5 Commercial Tools for Data Science

In the lecture on "Commercial Tools for Data Science," you learned about various categories of tools essential for data science tasks:

- **Data Management Tools**: Industry-standard databases, such as **Oracle Database**, **Microsoft SQL Server**, and **IBM Db2**, are crucial for storing enterprise data.
- **Data Integration Tools**: Tools like **Informatica PowerCenter** and **IBM InfoSphere DataStage** lead the market for ETL (Extract, Transform, Load) processes, helping design and deploy data processing pipelines.
- **Data Visualization Tools**: Business intelligence tools such as **Tableau**, **Microsoft Power BI**, and **IBM Cognos Analytics** are used to create visual reports and dashboards.
- **Model Building Tools**: **SPSS Modeler** and **SAS Enterprise Miner** are prominent tools for machine learning, with SPSS Modeler also available in Watson Studio Desktop.
- **Model Deployment and Monitoring**: Tools like **SPSS Collaboration and Deployment Services** facilitate model deployment, while model monitoring is still developing, often relying on open-source solutions.
- **Data Asset Management**: Tools from **Informatica** and **IBM** help manage data governance and lineage, ensuring data is versioned and compliant with regulations.
- **Integrated Development Environments**: **Watson Studio** and **H2O Driverless AI** provide comprehensive environments for data science tasks, covering the entire data science lifecycle.

6 Cloud-Based Tools for Data Science

6.1 Overview

Cloud-based tools support data science tasks by integrating multiple functions into a single platform. This integration enables seamless workflows across different stages of data science, machine learning, and artificial intelligence (AI) development.

6.2 Fully Integrated Visual Tools

Cloud-based visual tools provide a platform where large-scale execution of data science workflows occurs in compute clusters, which consist of multiple servers operating transparently in the background. Some prominent examples include:

- **Watson Studio and Watson OpenScale**: These platforms cover the complete development lifecycle for data science, machine learning, and AI tasks.
- **Microsoft Azure Machine Learning**: A full cloud-hosted service supporting the entire AI development process.
- **H2O Driverless AI**: Though a downloadable product, it offers one-click deployment for standard cloud service providers.

Unlike Watson Studio and Azure Machine Learning, H2O Driverless AI does not include operations and maintenance by the cloud provider, differentiating it from Platform as a Service (PaaS) or Software as a Service (SaaS).

6.3 Data Management in the Cloud

Many open-source and commercial data management tools have SaaS versions operated by cloud providers. The provider manages tasks such as data backup, configuration, and software updates. Some proprietary tools are exclusive to specific cloud providers, such as:

- **Amazon DynamoDB**: A NoSQL database that stores and retrieves data in keyvalue or document formats, typically using JSON.
- **Cloudant**: A database-as-a-service based on Apache CouchDB, offering automated operational tasks like backup and scaling while maintaining compatibility with CouchDB.
- **IBM Db2 as a Service**: A commercial database provided as a SaaS offering, eliminating user-side operational tasks.

6.4 Commercial Data Integration Tools

Data integration tools streamline Extract, Transform, Load (ETL) and Extract, Load, Transform (ELT) processes, shifting transformation tasks to data scientists and engineers. Widely used tools include:

- **Informatica Cloud Data Integration**
- **IBM Data Refinery** (part of Watson Studio), which transforms raw data into highquality, structured information.

6.5 Cloud-Based Data Visualization Tools

The market for cloud-based data visualization tools is vast, with each major cloud provider offering a solution. Some examples include:

- **Datameer**: A cloud-based data visualization tool from a smaller provider.
- **IBM Cognos Business Intelligence Suite**: Available as a cloud-based solution.
- **IBM Data Refinery**: Offers data exploration and visualization features within Watson Studio.

6.6 Common Data Visualization Techniques

- **3D Bar Chart**: Displays a target value along the vertical axis dependent on two horizontal values.
- **Hierarchical Edge Bundling**: Depicts correlations and affiliations between entities.
- **2D Scatter Plot with Heat Map**: Shows two dependent data fields using color intensities.
- **Tree Map**: Represents distribution within a dataset.
- **Pie Chart**: Displays non-hierarchical distributions.
- **Word Cloud**: Highlights significant terms within a document corpus.

6.7 Model Building and Deployment

Cloud-based services facilitate model training and deployment:

- **Watson Machine Learning**: Supports training and model building using opensource libraries.
- **Google AI Platform Training**: A similar cloud-based model training service.

- **SPSS Collaboration and Deployment Services**: Enables model deployment for SPSS software suite users.
- **Predictive Model Markup Language (PMML)**: A format used by SPSS Modeler for interoperability between commercial and open-source tools.
- **Watson Machine Learning (Deployment)**: Provides model deployment via REST APIs.
- **Amazon SageMaker Model Monitor**: A cloud-based tool for continuous monitoring of machine learning models.
- **Watson OpenScale**: Another model monitoring tool ensuring model performance and fairness.

6.8 Summary

- Watson Studio and Watson OpenScale for AI lifecycle management.
- SaaS versions of data management tools.
- Commercial data integration tools like Informatica Cloud Data Integration and IBM Data Refinery.
- Cloud-based data visualization solutions such as Datameer and IBM Cognos.
- Cloud services for model building and deployment, including Watson Machine Learning and Amazon SageMaker Model Monitor.

These cloud-based tools provide integrated solutions for efficient data science workflows.

7 Summary: Open Source Tools for Data Science

7.1 Objective

This reading provides a summary of key open-source tools for Data Science covered in the Part 1 and Part 2 videos of this course.

They are broadly classified as -

- **Data Management Tools** - Facilitates the storage, organization, and retrieval of data. Includes Relational Databases, NoSQL Databases, and Big Data platforms.
- **Data Integration and Transformation Tools** - Streamlines data pipelines and automate data processing workflows. Task of data integration and transformation in the classic data warehousing world is to Extract, Transform, and Load (ETL).
- **Data Visualization Tools**- Provides graphical representation of data and assist with communicating insights.
- **Model Deployment, Monitoring and Assessment Tools**- Supports the building, deploying, monitoring, and evaluation of data and machine learning models.
- **Data Asset Management Tools**- Organizes and manages data, enforce access controls, and ensure asset backups.
- **Code Development and Execution Tools** - ProvideS environments for developing, testing, and deploying code, offering computational resources to execute it.
- **Code Asset Management Tools** - Enables the storage and management of code, track changes, and support collaborative development.

7.2 Data Management Tools

7.2.1 MySQL

- Popular open source **relational database management system** (RDBMS)
- Uses structured query language (SQL) to manage and store data.
- Common uses:
  - Web applications
  - Data warehousing
  - E-commerce

7.2.2 PostgreSQL

- Powerful and open source **relational database management system** (RDBMS)
- Emphasizes extensibility and SQL compliance.
- Offers advanced features such as:
  - Support for JSON
  - Full-text search
  - Spatial data

7.2.3 Apache CouchDB

- Document-oriented **NoSQL** database
- Uses JSON to store data
- Highly scalable
- Fault-tolerant
- Easy to use

7.2.4 MongoDB

- Document-oriented **NoSQL** database • Stores data in a flexible JSON
- Provides:
  - Scalability
  - High availability
  - Data distribution
- Suitable for modern web applications that handle large volumes of unstructured data

7.2.5 Apache Cassandra

- Highly scalable, distributed Document-oriented **NoSQL** database
- Can handle large amounts of structured and unstructured data across many commodity servers.
- Offers:
  - High availability
  - Fault tolerance
  - Tunable consistency levels

- Suitable for mission-critical applications

7.2.6 Hadoop Distributed File System (HDFS)

- Designed to work with large datasets like Apache Hadoop in a distributed computing environment
- High-throughput data processing by splitting files into blocks (default 128MB), and these blocks are distributed across multiple DataNodes
- Data is replicated across different DataNodes ensuring high availability and fault tolerance
- Scalable and efficient

7.2.7 Ceph

- Free, open source software-defined storage platform suitable for hybrid cloud environments
- Designed for modern data centers
- Provides highly scalable, unified storage system that can be used for object storage (like AWS S3), block storage (like virtual disks for VMs), and file storage (like NFS) under one unified system
- High performance, availability and reliability

7.2.8 Elasticsearch

- Primarily a distributed RESTful search engine and analytics tool
- Based on the Lucene library.
- Full-text search, real-time data analytics
- Highly scalable
- Easy to use
- Powerful querying capabilities
- Real-time data indexing for fast document retrieval.

7.3 Data Integration and Transformation Tools

7.3.1 Apache Airflow

- Open-source platform for programmatically authoring, scheduling, and monitoring workflows
- Created originally by Airbnb
- Allows users to define and execute complex workflows • Support for: o Task dependencies o Parallelism o Error handling

7.3.2 Kubeflow

- An open-source machine learning toolkit that allows execution of data science pipelines on top of Kubernetes.

- Provides a platform for building, deploying, and managing end-to-end machine learning workflows at scale
- Support for:
    - Distributed training
    - Model serving
    - Hyperparameter tuning

7.3.3 Apache Kafka

- Distributed streaming platform that allows applications to publish, process, and subscribe to streams of records in real-time
- Created originally from LinkedIn.
- It is scalable, fault-tolerant, and high-throughput
- Suitable for building mission-critical, data-intensive applications

7.3.4 Apache NiFi

- An open-source data integration platform that allows users to automate the flow of data between systems
- Provides a web-based user interface for designing and managing data flows
- Support for:
    - Data routing
    - Transformation
    - Enrichment
    - Among other capabilities

7.3.5 Apache Spark SQL

- A module in the Spark ecosystem that provides a programming interface for working with structured data using:
    - SQL
    - Data frames
    - Datasets
- Supports a wide range of data sources and provides optimized performance for complex data processing tasks.

7.3.6 Node-RED

- An open-source visual programming tool for wiring together hardware devices, APIs, and online services
- Allows users to create event-driven flows of messages
- low in resource consumption that it even runs on tiny devices like a Raspberry Pi.
- Support for:
    - Data transformation
    - Filtering
    - Aggregation

7.4 Data Visualization Tools

7.4.1 Pixie Dust

- Open-source library for creating interactive, exploratory data visualizations in Python and Jupyter notebooks
- Provides a range of built-in visualizations and data connectors
- Support for customization and extensibility through third-party libraries

7.4.2 Hue

- Open-source web interface for analyzing and visualizing large datasets in Apache Hadoop
- Offers a user-friendly experience for exploring data and creating visualizations
- No need for programming skills; can create visualizations from SQL queries

7.4.3 Kibana

- Open-source data visualization tool that allows users to interact with their data through a web-based interface
- Commonly used with Elasticsearch to analyze and visualize large datasets

7.4.4Apache Superset

- A modern, enterprise-ready business intelligence web application that makes it easy to visualize and explore large datasets
- Offers a rich set of data visualization options, including:
    - Charts o Tables
    - Maps
    - Geospatial analysis o Real-time data processing

7.5 Model Deployment Tools

7.5.1 Apache PredictionIO

- Open-source machine learning server built on a scalable and distributed infrastructure
- Allows developers to quickly build, evaluate, and deploy predictive engines for various use cases such as: o Recommendation o Classification o Clustering

7.5.2 Kubernetes

- Open-source platform for container orchestration

- Automatically launches, scales, and manages containerized applications • Offering features like:
    - Automatic scaling
    - Self-healing o Load balancing
- Enables the management and orchestration of containers across numerous hosts

7.5.3 Apache Seldon

- Open-source platform for deploying and managing machine learning models on Kubernetes
- Provides a way to:
    - Serve models at scale
    - Automate model deployment workflows
    - Monitor the performance of deployed models in real-time

7.5.4 MLeap

- Open-source library for serializing and deserializing learning models in a crossplatform file
- Gives users the ability to export models from different machine learning libraries and frameworks, such as: o Spark
    - Scikit-learn o TensorFlow
- Implements them in high-throughput, low-latency production environments

7.5.5 TensorFlow Lite

- Open-source tool for running machine learning models on mobile and embedded devices
- Allows effective inference on mobile and embedded platforms • Supports a variety of hardware accelerators such as: o CPUs o GPUs o Custom ASICs

7.5.6 Red Hat OpenShift

- Container application framework based on Kubernetes
- With characteristics like automation, scalability, and security
- Offers a method for creating, deploying, and managing containerized applications

7.5.7 TensorFlow Serving

- Open-source utility that serves machine learning models in real-world settings
- Supports both HTTP and gRPC interfaces for serving predictions

- Provides high scalability and low latency deployment and management of TensorFlow models

7.5.8 TensorFlow.js

- Open-source library for building and deploying machine learning models in JavaScript
- Allows you to train and execute models directly in the browser or on Node.js
- Supports a wide range of model architectures, including neural networks, decision trees, and k-nearest neighbors

7.6 Model Monitoring and Assessment Tools

7.6.1 ModelDB

- Open-source platform for managing machine learning models and experiments
- Provides a way to track and reproduce experiments, version models, and collaborate with team members

7.6.2Prometheus

- Freely available monitoring system that collects and stores metrics in real-time from different sources
- Allows you to visualize and set alerts on the health and performance of systems and apps
- Supports a variety of data gathering methods, such as HTTP endpoints, exporters, and agents

7.6.3 IBM AI Fairness 360

- Open-source toolkit for detecting and mitigating bias in machine learning models
- Provides a way to measure the fairness and bias of models, as well as a set of algorithms for mitigating bias and creating fairer models

7.6.4 IBM AI Explainability 360

- Open-source toolkit for explaining the behavior and decisions of machine learning models
- Provides a way to measure the explainability and interpretability of models, as well as a set of algorithms for generating explanations and visualizations of model behavior

7.6.5 IBM Adversarial Robustness 360 Toolbox

- Free and open-source library for protecting machine learning models from adversarial attacks
- Includes a method for measuring model robustness and vulnerability

- Includes a set of algorithms for improving model robustness and detecting adversarial examples

7.7 Code Development and Execution Tools

7.7.1 Jupyter IDE

- Open-source effort • Supports:
  - Julia o Python
  - R development with Jupyter Notebook o JupyterLab o JupyterHub
- Create and share documents containing:
  - Live code o Equations
  - o Visualizations o Narrative text
- JupyterLab includes:
  - Customized notebook organization
- JupyterHub extends all these capabilities to the enterprise

7.7.2 RStudio

- For developers
- Free and open-source IDE
- Built to manage and execute R code • Works on all platforms
- Includes:
  - Version control
  - Project management capabilities

7.7.3 Microsoft Visual Studio

- An IDE that supports a variety of programming languages, including: o C o C++ o C++/CLI o Visual Basic.NET
  - C#
  - F#
  - JavaScript
  - TypeScript
  - XML o XSLT o HTML o CSS
- Using plug-ins, supports:
  - Python o Ruby
  - Node.js
  - M
  - Other languages

7.7.4 PyCharm

- Primarily a subscription-based IDE environment
- Offers 16+ additional tools for coding assistance, testing, and web development
- Supports scientific development with IPython integration and Matplotlib and NumPy support
- Also offers a free community-based, open-source IDE with limited capabilities

7.7.5Spyder

- Free, open-source Python-based IDE designed by and for scientists, engineers, and data analysts
- Features a unique combination of comprehensive development tools for:
  - Advanced editing o Analysis o Debugging o Profiling
  - Visualization capabilities

7.7.6 Anaconda Navigator

- Open-source GUI-based Navigator that supports Python development and integrates with:
  - Eclipse and PyDev o IDLE
  - IntelliJ
  - Microsoft Visual Studio Code (VS Code)
  - Ninja IDE
  - PyCharm
  - Python for Visual Studio Code
  - Python Tools for Visual Studio (PTVS)
  - Spyder o Sublime Text o Wing IDE

7.8 Code Asset Management Tools

7.8.1 Git

- Open-source version control system for tracking changes in code and collaboration among developers
- Provides a way to manage and organize code changes, collaborate on code development, and maintain a history of code revisions

7.8.2 GitLab

- Web-based Git repository manager
- Provides a complete DevOps platform for: o Source code management o Continuous integration and deployment o Monitoring
- Enables teams to collaborate on:
  - Code development
  - Automate build and deployment processes

- Track metrics and performance across the entire software development lifecycle

7.8.3 GitHub

- Web-based Git repository hosting service that provides a platform for developers to collaborate on code and manage software projects
- Enables users to:
  - Create, fork, and contribute to open source projects
  - Track changes in code o Manage issues o Pull requests

7.8.4 Bitbucket from Atlassian

- Web-based Git repository hosting service
- Provides a platform for developers to collaborate on code and manage software projects, with features like:
  - Pull requests o Code review o Branch permissions

8 Module 1 Summary

- The Data Science Task Categories include:
  - Data Management - storage, management and retrieval of data o Data Integration and Transformation - streamline data pipelines and automate data processing tasks
  - Data Visualization - provide graphical representation of data and assist with communicating insights
  - Modelling - enable Building, Deployment, Monitoring and Assessment of Data and Machine Learning models
- Data Science Tasks support the following:
  - Code Asset Management - store & manage code, track changes and allow collaborative development
  - Data Asset Management - organize and manage data, provide access control, and backup assets
  - Development Environments - develop, test and deploy code o Execution Environments - provide computational resources and run the code

The data science ecosystem consists of many open source and commercial options, and include both traditional desktop applications and server-based tools, as well as cloudbased services that can be accessed using web-browsers and mobile interfaces.
Data Management Tools: include Relational Databases, NoSQL Databases, and Big Data platforms:

- MySQL, and PostgreSQL are examples of Open Source Relational Database Management Systems (RDBMS), and IBM Db2 and SQL Server are examples of commercial RDBMSes and are also available as Cloud services.

- MongoDB and Apache Cassandra are examples of NoSQL databases.
- Apache Hadoop and Apache Spark are used for Big Data analytics.

Data Integration and Transformation Tools: include Apache Airflow and Apache Kafka.

Data Visualization Tools: include commercial offerings such as Cognos Analytics, Tableau and PowerBI and can be used for building dynamic and interactive dashboards.

Code Asset Management Tools: Git is an essential code asset management tool. GitHub is a popular web-based platform for storing and managing source code. Its features make it an ideal tool for collaborative software development, including version control, issue tracking, and project management.

Development Environments: Popular development environments for Data Science include Jupyter Notebooks and RStudio.

- Jupyter Notebooks provides an interactive environment for creating and sharing code, descriptive text, data visualizations, and other computational artifacts in a web-browser-based interface.
- RStudio is an integrated development environment (IDE) designed specifically for working with the R programming language, which is a popular tool for statistical computing and data analysis.

Module 2

9 Languages of Data Science

In the current lecture of the "Languages of Data Science" module, you learned about:

- **Choosing a Programming Language**: The selection of a programming language depends on your specific needs, the problems you aim to solve, and the context of your work. Recommended languages include **Python**, **R**, and **SQL**, with others like **Scala**, **Java**, **C++**, and **Julia** also being popular for specific tasks.
- **Roles in Data Science**: Various roles in the data science field include:
    - Business Analyst
    - Database Engineer
    - Data Analyst
    - Data Engineer
    - Data Scientist
    - Research Scientist
    - Software Engineer
    - Statistician
    - Product Manager
    - Project Manager

Understanding the problems you need to address will guide your choice of programming language and the role you might pursue in data science.

10 Introduction to Python

10.1 Users of Python

Python is widely used by various professionals due to its clear and readable syntax. It is beneficial for both experienced programmers and beginners:

10.1.1 Experienced Programmers

- Python allows them to develop the same programs with less code compared to other languages.

10.1.2 Beginners

- Python is a great starting language because of its vast global community and extensive documentation.

10.2 Benefits of Using Python

Python offers numerous advantages, making it one of the most popular programming languages:

10.2.1 General Features

- It is a high-level, general-purpose programming language.
- It provides a large standard library suitable for various tasks, including:
    - Databases
    - Automation
    - Web scraping
    - Text processing
    - Image processing
    - Machine learning
    - Data analytics

10.2.2 Applications of Python

- It is widely used in fields such as:
    - **Data Science**
    - **Artificial Intelligence and Machine Learning**
    - **Web Development**
    - **Internet of Things (IoT)**
- It is supported by a vast global community, ensuring continued development and improvement.

10.3 Popularity of Python

Python is a dominant language in the tech industry:

- According to the 2019 Kaggle Data Science and Machine Learning Survey, 75% of over 10,000 respondents reported using Python regularly.
- Glassdoor stated that in 2019, more than 75% of data science job listings included Python as a required skill.
- Over 80% of data professionals worldwide use Python, as reported by multiple 2019 surveys.

10.3.1 Organizations Using Python

Many leading organizations utilize Python, including:

- IBM
- Wikipedia • Google
- Yahoo!
- CERN
- NASA
- Facebook
- Amazon
- Instagram
- Spotify
- Reddit

10.4 Python in Data Science and Artificial Intelligence

Python is extensively used in Data Science and AI due to its powerful libraries:

10.4.1 Scientific Computing Libraries

- Pandas
- NumPy
- SciPy
- Matplotlib

10.4.2 Artificial Intelligence and Machine Learning

- TensorFlow
- PyTorch
- Keras
- Scikit-learn

10.4.3 Natural Language Processing (NLP)

- Natural Language Toolkit (NLTK)

10.5 Diversity and Inclusion in the Python Community

The Python community actively promotes diversity and inclusion:

10.5.1 Python Software Foundation

- Enforces a code of conduct to ensure safety and inclusivity in both online and inperson communities.

10.5.2 PyLadies

- An international mentorship group focusing on encouraging women to become active participants and leaders in the Python open-source community.

10.6 Summary

- Python has a clear and readable syntax, making it suitable for both beginners and experienced programmers.
- It is widely used in Data Science, AI, and Web Development, supported by powerful libraries.
- Python has a strong global community and extensive documentation.
- The Python community promotes diversity and inclusion through initiatives like the Python Software Foundation and PyLadies.

11 Introduction to R Language

11.1 Open Source vs. Free Software

Understanding the distinction between open source and free software is essential when working with programming languages like R and Python.

11.1.1 Similarities

- Both are free to use.
- Both commonly refer to the same set of licenses, such as the General Public License (GNU).
- Both promote collaboration.
- In many cases, the terms can be used interchangeably (but not always).

11.1.2 Differences

- The **Open-Source Initiative (OSI)** champions open source, while the **Free Software Foundation (FSF)** defines free software.
- Open source is more business-focused, whereas free software emphasizes a set of values.

11.2 Why Learn R?

R is a powerful language for statistical computing and data analysis with multiple benefits:

- **Free Software**: It allows private use, commercial use, and public collaboration.
- **Global Community Support**: R has a large community that contributes to solving complex problems.

- **Ideal for Statisticians & Data Scientists**: Used for developing statistical software, graphing, and data analysis.
- **User-Friendly Syntax**: R's array-oriented syntax makes it easier for learners with minimal programming background to transition from mathematics to coding.

11.3 Popularity of R

- According to the **2019 Kaggle Data Science Survey**, learning R along with other programming languages can increase earning potential.
- Most programmers learn R after gaining experience in data science.
- R is widely used in **academia** and industry.

11.3.1 Organizations Using R

Leading companies that use R include:

- IBM
- Google
- Facebook
- Microsoft
- Bank of America
- Ford
- TechCrunch
- Uber
- Trulia

11.4 Capabilities of R

R is considered the world's **largest repository of statistical knowledge** with more than **15,000 publicly released packages** (as of 2018). These packages enable advanced exploratory data analysis and complex statistical modeling.

11.4.1 Integration with Other Languages

R integrates seamlessly with:

- **C++**
- **Java**
- **C**
- **.NET**
- **Python**

11.4.2 Mathematical Operations

- Supports advanced mathematical operations like **matrix multiplication** with immediate results.

- Offers **stronger object-oriented programming (OOP) capabilities** compared to most statistical computing languages.

11.5 Connecting with the R Community

To engage with the global R community, consider joining these groups:

- useR
- WhyR
- SatRdays
- R-Ladies
- The R Project website for conferences and events.

11.6 Summary

- Python is open source, while R is free software.
- The Open-Source Initiative (OSI) and Free Software Foundation (FSF) govern these respective concepts.
- R's syntax makes it easier for non-programmers to transition into coding.
- R has become the **world's largest repository of statistical knowledge** with thousands of packages for data analysis.
- The global R community supports learning and collaboration through various groups and events.

12 Introduction to SQL

12.1 Understanding SQL and Relational Databases

SQL (Structured Query Language) is a non-procedural language designed for managing and querying data in relational databases. Unlike procedural programming languages, SQL focuses on handling structured data, which involves relationships between entities and variables.

12.1.1 Key Characteristics of SQL

- Officially pronounced as **"ess cue el"**, though some call it **"sequel"**.
- First appeared in 1974 and was developed at **IBM**.
- Older than **Python and R** by about 20 years.
- Commonly used by **data scientists** due to its simplicity and power.
- Designed primarily for **relational databases** but also interfaces with **NoSQL and big data repositories**.

12.2 Structure of a Relational Database

A relational database consists of collections of **two-dimensional tables**, similar to datasets or Excel spreadsheets:

- **Tables** contain a fixed number of **columns** and an unlimited number of **rows**.

- **SQL's pervasiveness and ease of use** have led to its integration into various **NoSQL** and **big data repositories**.

12.3 Elements of SQL

SQL is subdivided into several language elements:

- **Clauses** – Components of SQL statements.
- **Expressions** – Produce values from stored data.
- **Predicates** – Used for conditions and filtering.
- **Queries** – Retrieve data from one or more tables.
- **Statements** – Perform operations such as inserting, updating, or deleting data.

12.4 Benefits of Using SQL

SQL provides numerous advantages that make it essential in data-related roles:

12.4.1 Key Benefits

- Essential for jobs in **Data Science, Business Analysis, and Data Engineering**.
- Directly accesses data **without requiring separate copying**, improving workflow speed.
- Acts as an **interpreter between the user and the database**.
- Recognized as an **ANSI standard**, allowing SQL knowledge to be transferable across different database systems.

12.5 Popular SQL Databases

Various relational database management systems (RDBMS) implement SQL, including:

- **MySQL**
- **IBM DB2**
- **PostgreSQL**
- **Apache Open Office Base**
- **SQLite**
- **Oracle**
- **MariaDB**
- **Microsoft SQL Server**

12.6 Learning SQL Effectively

Since SQL syntax may vary across RDBMS platforms, it is beneficial to:

- Focus on learning SQL within a **specific database system**.
- Engage with the **community of that platform**.
- Explore **introductory courses** to build a strong foundation.

12.7 Summary

- SQL is a **non-procedural** language focused on querying and managing data.
- It was designed for **relational databases** but is now widely used across various database types.
- SQL behaves as an **interpreter** between users and databases.
- **Learning SQL** in one database system allows easy application to many others.

13 Other Languages for Data Science

13.1 Overview

Previously, we reviewed Python, R, and SQL. In this lesson, we will examine additional languages with compelling use cases in data science. Scala, Java, C++, and Julia are among the most traditional data science languages. However, JavaScript, PHP, Go, Ruby, Visual Basic, and others have also found a place in the data science community.

13.2 Java

Java is a general-purpose, object-oriented programming language widely adopted in the enterprise space. It is designed for speed and scalability. Java applications are compiled to bytecode and run on the Java Virtual Machine (JVM).

13.2.1 Data Science Tools in Java

- **Weka** – Data mining
- **Java-ML** – Machine learning
- **Apache MLlib** – Scalable machine learning
- **Deeplearning4** – Deep learning
- **Hadoop** – Manages data processing and storage for big data applications

13.3 Scala

Scala is a general-purpose programming language that supports functional programming and features a strong static type system. It was created to address Java's shortcomings and runs on the JVM.

13.3.1 Data Science Tools in Scala

- **Apache Spark** – A fast and general-purpose cluster computing system
  - **Shark** – Query engine
  - **MLlib** – Machine learning
  - **GraphX** – Graph processing
  - **Spark Streaming** – Real-time data processing

Designed to be faster than Hadoop

13.4 C++

C++ is an extension of the C programming language, providing better processing speed, system programming capabilities, and greater control over software applications. Many organizations use C++ for real-time data applications while using Python for analysis.

13.4.1 Data Science Tools in C++

- **TensorFlow** – A deep learning library for dataflow (Python interface available)
- **MongoDB** – A NoSQL database for big data management
- **Caffe** – Deep learning algorithm repository (Python and MATLAB bindings)

13.5 JavaScript

JavaScript is a general-purpose programming language that extended beyond the browser with the creation of Node.js and other server-side frameworks. JavaScript is **not** related to Java.

13.5.1 Data Science Tools in JavaScript

- **TensorFlow.js** – Enables machine learning and deep learning in Node.js and the browser
- **Brain.js & machinelearn.js** – Open-source JavaScript machine learning libraries
- **R-js** – Translates linear algebra specifications from R into TypeScript, paving the way for math frameworks similar to NumPy and SciPy

13.6 Julia

Julia was designed at MIT for high-performance numerical analysis and computational science. It offers the ease of development found in Python or R while achieving speeds comparable to C or Fortran. Julia supports refined parallelism and can call C, Go, Java, MATLAB, R, Fortran, and Python libraries.

13.6.1 Data Science Tools in Julia

- **JuliaDB** – A package for handling large, persistent datasets

13.7 Key Takeaways

- **Java**: Weka, Java-ML, Apache MLlib, and Deeplearning4.
- **Scala**: Apache Spark (Shark, MLlib, GraphX, Spark Streaming).
- **C++**: TensorFlow, MongoDB, Caffe.
- **JavaScript**: TensorFlow.js, R-js.
- **Julia**: JuliaDB.

14 Module 2 Summary

- You should select a language to learn depending on your needs, the problems you are trying to solve, and whom you are solving them for.

- The popular languages are Python, R, SQL, Scala, Java, C++, and Julia.
- For data science, you can use Python's scientific computing libraries like Pandas, NumPy, SciPy, and Matplotlib.
- Python can also be used for Natural Language Processing (NLP) using the Natural Language Toolkit (NLTK).
- Python is open source, and R is free software.
- R language's array-oriented syntax makes it easier to translate from math to code for learners with no or minimal programming background.
- SQL is different from other software development languages because it is a nonprocedural language.
- SQL was designed for managing data in relational databases.
- If you learn SQL and use it with one database, you can apply your SQL knowledge with many other databases easily.
- Data science tools built with Java include Weka, Java-ML, Apache MLlib, and Deeplearning4.
- For data science, popular program built with Scala is Apache Spark which includes Shark, MLlib, GraphX, and Spark Streaming.
- Programs built for Data Science with JavaScript include TensorFlow.js and R-js. • One great application of Julia for Data Science is JuliaDB.

Module 3

15 Libraries for Data Science

15.1 Overview

After completing this section, you will be able to:

- List the scientific computing libraries in Python.
- List the visualization libraries in Python.
- List the high-level machine learning and deep learning libraries.
- List the libraries used in other languages.

Libraries are collections of functions and methods that allow you to perform various actions without writing the code from scratch. This section focuses on different Python libraries and their applications.

15.2 Scientific Computing Libraries in Python

Scientific computing libraries contain built-in modules that provide various functionalities, also known as frameworks.

15.2.1 Pandas

- Offers data structures and tools for data cleaning, manipulation, and analysis.

- The primary data structure is a **DataFrame**, a two-dimensional table consisting of columns and rows.
- Provides easy indexing for efficient data handling.

15.2.2 NumPy

- Based on arrays and matrices.
- Allows mathematical functions to be applied to arrays.
- Pandas is built on top of NumPy.

15.3 Visualization Libraries in Python

Data visualization libraries enable the creation of graphs, charts, and maps to display meaningful results from an analysis.

15.3.1 Matplotlib

- The most well-known library for data visualization.
- Used for creating customizable graphs and plots.

15.3.2 Seaborn

- Built on top of Matplotlib.
- Used for generating heat maps, time series, and violin plots.

15.4 High-Level Machine Learning and Deep Learning Libraries

These libraries provide simplified interfaces for building machine learning and deep learning models without dealing with low-level details.

15.4.1 Scikit-learn

- Contains tools for statistical modeling, including regression, classification, and clustering.
- Built on **NumPy, SciPy,** and **Matplotlib**.
- Provides an easy-to-use high-level approach to defining models and specifying parameters.

15.4.2 Keras

- Allows quick and simple deep learning model building.
- Supports Graphics Processing Units (GPUs) for computation.
- Functions as a high-level interface for deep learning frameworks.

15.5 Deep Learning Libraries in Python

15.5.1 TensorFlow

- A low-level framework designed for large-scale deep learning model production and deployment.
- Can be complex for experimentation but is powerful for large-scale applications.

15.5.2 PyTorch

- Used for deep learning research and experimentation.
- Preferred by researchers for testing new ideas.

15.6 Libraries Used in Other Languages

Some data science and machine learning libraries exist outside of Python.

15.6.1 Apache Spark

- A general-purpose **cluster-computing framework**.
- Enables parallel data processing across multiple computers.
- Provides similar functionality to **Pandas, NumPy,** and **Scikit-learn**.

15.6.1.1 Supported Languages for Spark Jobs

- **Python**
- **R**
- **Scala**
- **SQL**

15.6.2 Scala Libraries

Scala is widely used in **data engineering** and **data science**.

15.6.2.1 Vegas

- A Scala library for **statistical data visualization**.
- Works with **data files** and **Spark DataFrames**.

15.6.2.2 BigDL

- A deep learning library for **Scala**.

15.6.3 R Libraries

R is widely used for **machine learning** and **data visualization**.

15.6.3.1 ggplot2

- A popular library for **data visualization** in R.

15.6.3.2 Integration with Python Libraries

- R supports libraries that interface with **Keras** and **TensorFlow**.

16 Application Programming Interfaces (APIs)

16.1 Definition of API

An Application Programming Interface (API) allows communication between two pieces of software. It acts as an intermediary that enables data exchange and functionality integration without requiring knowledge of the back-end implementation.

16.1.1 How APIs Work

- APIs facilitate interaction between software components using inputs and outputs.
- They expose only the interface while containing various program components.
- APIs allow different programming languages to interact with the same back-end components.

16.2 API Libraries

APIs are commonly used in libraries to provide structured access to functionalities.

16.2.1 Example: Pandas Library

- Pandas is a collection of software components used for data processing.
- Not all components in Pandas are written in Python.
- The Pandas API allows users to process data by interacting with other software components.

16.2.2 Example: TensorFlow API

- TensorFlow is written in C++ but provides APIs for various languages, including:
  - Python
  - JavaScript o C++ o Java o Go
- Additionally, volunteer-developed APIs exist for:
  - Julia
  - MATLAB o R o Scala o Others

16.3 REST API (Representational State Transfer API)

REST APIs are a widely used type of API that enable communication over the internet.

16.3.1 Features of REST APIs

- REST APIs allow access to resources such as:

- Storage
- Data
- AI algorithms
- Other web services

- They use standard HTTP methods to transmit data between a client and a web service.

16.3.2 Common REST API Terminology

- **Client:** The user or application making a request.
- **Resource:** The web service providing data or functionality.
- **Endpoint:** The URL through which the client accesses the resource.
- **Request:** The operation performed by the client (sent via HTTP message in JSON format).
- **Response:** The data returned by the web service (sent via HTTP message in JSON format).

16.3.3 How REST APIs Work

1. The client sends a request to an endpoint using an HTTP method.
2. The request is formatted as a JSON file specifying the desired operation.
3. The web service processes the request and performs the operation.
4. The web service returns a response in JSON format to the client.

16.3.4 Example: Watson Speech-to-Text API

- Converts spoken words into text.
- The client sends an audio file to the API (POST request).
- The API processes the file and returns the transcribed text (GET request at the back end).

16.3.5 Example: Watson Language Translator API

- Translates text from one language to another.
- The client sends a text input to the API.
- The API translates the text and returns the translated version.
- Example: English to Spanish translation.

16.4 Summary

- APIs facilitate communication between software components.
- API libraries, such as Pandas and TensorFlow, provide structured access to functionalities.
- REST APIs enable communication over the internet, using HTTP methods and JSON data exchange.
- Watson APIs provide practical examples of REST API applications.
- Open-source and free software initiatives promote accessibility and collaboration.
- R language is widely used for statistical computing and has an active global community.

17 Data Sets – Powering Data Science

17.1 Defining a Data Set

A data set is a structured collection of data that embodies information represented as text, numbers, or media such as images, audio, or video files.

17.1.1 Tabular Data

- A tabular data set consists of rows and columns storing information.
- A popular format for tabular data is **Comma-Separated Values (CSV)**.
- In a CSV file, each line represents a row, and a comma separates data values.
- Example: A weather station dataset where each row represents an observation, and each column contains details like temperature, humidity, and other weather conditions.

17.1.2 Hierarchical and Network Data

- **Hierarchical Data** is organized in a tree-like format.
- **Network Data** is stored as a graph, often used to represent relationships (e.g., social networking connections).
- A data set might also include raw data files such as images or audio.
- Example: The **MNIST dataset**, which contains images of handwritten digits, is widely used in image processing and machine learning.

17.2 Types of Data Ownership

Data ownership can be categorized into private and open data.

17.2.1 Private Data

- Contains proprietary or confidential information (e.g., customer data, pricing data).
- Typically not shared publicly.

17.2.2 Open Data • Shared by governments, organizations, and companies for public use.

- Enables researchers and analysts to uncover insights and create new applications.
- **Examples of Open Data Providers:** o The United Nations
  - Federal and municipal governments o Scientific institutions
  - Organizations like the European Union

17.3 Sources of Data

Many sources provide open data for public use:

- **Open Knowledge Foundation's datacatalogs.org**: Lists global data portals.
- **United Nations and European Union repositories**: Provide access to vast datasets.

- **Kaggle**: An online data science community where users can find and contribute datasets.
- **Google Dataset Search**: A search engine for discovering valuable datasets.

17.4 Community Data License Agreement (CDLA)

The **Linux Foundation** created the **CDLA** to standardize open data distribution and licensing.

17.4.1 Types of CDLA Licenses

1. **CDLA-Sharing**:
   - Grants permission to use and modify data.
   - Requires modified data to be shared under the same license terms.
2. **CDLA-Permissive**:
   - Grants permission to use and modify data. o Does not require sharing modifications.

Both licenses ensure no restrictions on results derived from data. If a model is trained on CDLA-licensed data, there is no obligation to share the model or license it in a specific way.

17.5 Summary

- Open data plays a vital role in data science, enabling research and innovation.
- The **Community Data License Agreement (CDLA)** simplifies open data sharing.
- Open datasets may not always meet enterprise requirements due to business impact.

18 Reading: Additional Sources of Datasets

In this reading, you will learn about:

- Open datasets and sources
- Proprietary datasets and sources
- Dataset license

18.1 Open datasets and sources

In this data-driven world, some datasets are freely available for anyone to access, use, modify, and share. These are called **open datasets**.
Open datasets include a public license and are very useful for your journey as a Data Scientist. Some of the most informative open dataset sources are listed below.

Government Data:

- https://www.data.gov/
- https://www.census.gov/data.html
- https://data.gov.uk/

- https://www.opendatanetwork.com/
- https://data.un.org/ **Financial Data Sources:**

- https://data.worldbank.org/
- https://www.globalfinancialdata.com/
- https://comtrade.un.org/
- https://www.nber.org/
- https://fred.stlouisfed.org/

Crime Data:

- https://www.fbi.gov/services/cjis/ucr
- https://www.icpsr.umich.edu/icpsrweb/content/NACJD/index.html
- https://www.drugabuse.gov/related-topics/trends-statistics
- https://www.unodc.org/unodc/en/data-and-analysis/

Health Data:

- https://www.who.int/gho/database/en/
- https://www.fda.gov/Food/default.htm
- https://seer.cancer.gov/faststats/selections.php?series=cancer
- https://www.opensciencedatacloud.org/
- https://pds.nasa.gov/
- https://earthdata.nasa.gov/
- https://www.sgim.org/communities/research/dataset-compendium/publicdatasets-topic-grid

Academic and Business Data:

- https://scholar.google.com/
- https://nces.ed.gov/
- https://www.glassdoor.com/research/
- https://www.yelp.com/dataset **Other General Data:**

- https://www.kaggle.com/datasets
- https://www.reddit.com/r/datasets/

18.2 Propriety datasets and sources

Proprietary datasets contain data primarily owned and controlled by specific individuals or organizations. This data is limited in distribution because it is sold with a licensing agreement.
Some data from private sources cannot be easily disclosed, like public data.

National security data, geological, geophysical, and biological data are examples of propriety data. Copyright laws or patents usually bind this type of data. Proprietary datasets that mainly contain sensitive information are less widely available than open datasets.

Some standard propriety dataset sources are listed below.

Health Care: https://www.sgim.org/communities/research/dataset-compendium/proprietarydatasets

Financial Market data:

https://datarade.ai/data-categories/proprietary-market-data

Google Cloud based datasets: https://cloud.google.com/datasets

18.3 Dataset licenses

When you select a dataset, it is necessary to look into the license. A license explains whether you can use that dataset or not; or explains if you have to accept certain guidelines to use that dataset. The different license types are listed below.

1. **PUBLIC DOMAIN MARK - PUBLIC DOMAIN**
   When a dataset has a Public Domain license, all the rights to use, access, modify and share the dataset are open to everyone. Here there is technically no license.
2. **OPEN DATA COMMONS PUBLIC DOMAIN DEDICATION AND LICENSE – PDDL**
   Open Data Commons license has the same features as the Public Domain license, but the difference is the PDDL license uses a licensing mechanism to give the rights to the dataset.
3. **CREATIVE COMMONS ATTRIBUTION 4.0 INTERNATIONAL CC-BY**
   This license allows users to share and modify a dataset, but only if they give credit to the creator(s) of the dataset.
4. **COMMUNITY DATA LICENSE AGREEMENT – CDLA PERMISSIVE-2.0**
   Like most open-source licenses, this license allows users to use, modify, adapt, and share the dataset, but only if a disclaimer of warranties and liability is also included.
5. **OPEN DATA COMMONS ATTRIBUTION LICENSE - ODC-BY**
   This license allows users to share and adapt a dataset, but only if they give credit to the creator(s) of the dataset.
6. **CREATIVE COMMONS ATTRIBUTION-SHAREALIKE 4.0**

INTERNATIONAL - CC-BY-SA

This license allows users to use, share, and adapt a dataset, but only if they give credit to the dataset and show any changes or transformations, they made to the dataset. Users might not want to use this license because they have to share the work they did on the dataset.

7. **COMMUNITY DATA LICENSE AGREEMENT – CDLA-SHARING-1.0**

   This license uses the principle of 'copyleft': users can use, modify, and adapt a dataset, but only if they don't add license restrictions on the new work(s) they create with the dataset.

8. **OPEN DATA COMMONS OPEN DATABASE LICENSE - ODCODBL**

   This license allows users to use, share, and adapt a dataset but only if they give credit to the dataset and show any changes or transformations they make to the dataset. Users might not want to use this license because they have to share the work they did on the dataset.

9. **CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIAL 4.0 INTERNATIONAL - CC BY-NC**

   This license is a restrictive license. Users can share and adapt a dataset, provided they give credit to its creator(s) and ensure that the dataset is not used for any commercial purpose.

10. **CREATIVE COMMONS ATTRIBUTION-NO DERIVATIVES 4.0 INTERNATIONAL - CC BY-ND**

    This license is also a restrictive license. Users can share a dataset if they give credit to its creator(s). This license does not allow additions, transformations, or changes to the dataset.

11. **CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIALSHAREALIKE 4.0 INTERNATIONAL - CC BY-NC-SA**

    This license allows users to share a dataset only if they give credit to its creator(s). Users can share additions, transformations, or changes to the dataset, but they cannot use the dataset for commercial purposes.

12. **CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIALNODERIVATIVES 4.0 INTERNATIONAL - CC BY-NC-ND**

    This license allows users to share a dataset only if they give credit to its creator(s). Users are not allowed to modify the dataset and are not allowed to use it for commercial purposes.

Note: Additional license types exist. Any dataset you use will include details about its license.

19 Sharing Enterprise Data - Data Asset Exchange

19.1 Introduction to Data Asset Exchange (DAX)

IBM's Data Asset Exchange (DAX) is an open data repository designed to provide highquality datasets with clearly defined licenses and usage terms. It offers a curated collection of datasets from IBM Research and trusted third-party sources, making it easier for developers to access and utilize open data in enterprise applications.

19.2 Features of Data Asset Exchange

DAX includes datasets in various formats such as images, video, text, and audio. It promotes data sharing and collaboration through the **Community Data License Agreement (CDLA)**, ensuring that datasets remain accessible and properly licensed for enterprise use.

19.2.1 Benefits of Using DAX

- Provides high-quality datasets in one central location
- Includes tutorial notebooks for data cleaning, preprocessing, and exploratory analysis
- Offers advanced notebooks for tasks like:
  - Creating charts
  - Training machine learning models
  - Integrating deep learning via the **Model Asset Exchange (MAX)**
  - Conducting statistical and time-series analysis
- Available on the IBM Developer Website

19.3 Exploring Data Sets on DAX

DAX allows users to browse and access multiple open datasets. One example is the **NOAA Weather Data (JFK Airport Database)**, which contains weather data from John F. Kennedy Airport in New York.

19.3.1 Accessing and Using Data Sets

- **Download Data**: Click "Get this data set" to download from cloud storage.
- **Run Notebooks**: Access Jupyter notebooks related to the dataset in Watson Studio.
- **Preview Metadata & Glossary**: Explore dataset metadata, glossary, and additional notebook details.
- **View and Execute Notebooks**: Under the "Assets" section, users can:
  - View all Jupyter notebooks associated with a dataset
  - Access source code for deeper exploration
  - Execute notebooks in Watson Studio for data cleaning, preprocessing, and analysis

19.4 Working with DAX in Watson Studio

Users familiar with Watson Studio can log into their **IBM Cloud Account**, create a project, and load DAX notebooks into their workspace.

19.4.1 Managing Data Files

Each dataset on DAX consists of one or more data files. Users can click the **Data** option to view and manage available data files within their project.

19.5 Summary

- DAX provides high-quality, open datasets with clear licensing terms.
- It includes tutorial and advanced notebooks for developers.
- DAX and MAX are accessible through the IBM Developer Website.
- Users can preview, run, and execute datasets and notebooks in Watson Studio.
- Watson Studio enables seamless integration of DAX datasets into data projects.

By leveraging DAX, developers can create end-to-end analytics and machine learning workflows with confidence, using high-quality, trusted datasets.

20 Machine Learning Models – Learning from Models to Make Predictions

20.1 What is a Machine Learning Model?

A machine learning (ML) model is an algorithm designed to identify patterns in data and make predictions based on those patterns. The process of learning these patterns from data is known as **model training**.

20.1.1 Traditional Data Analysis vs. Machine Learning

Traditional data analysis approaches involve manual inspection by a person or specialized programs. However, these methods have limitations due to the sheer volume of data or the complexity of the problem. Machine learning automates pattern recognition, making datadriven predictions more efficient and scalable.

20.2 Types of Machine Learning Models

Machine learning models can be divided into three basic classes:

20.2.1 1. Supervised Learning

Supervised learning involves a human providing input data along with correct outputs. The model learns to identify relationships between input and output data.

20.2.1.1 Regression Models

Regression models predict a numeric (real) value. Example:

- Predicting house prices based on location, size, and features.

20.2.1.2 Classification Models

Classification models categorize data into distinct classes. Example:

- Identifying emails as spam or not spam.

20.2.2 2. Unsupervised Learning

In unsupervised learning, data is not labeled by humans. The model analyzes the data and identifies patterns or structures.

20.2.2.1 Clustering Models

Clustering models group similar records together. Examples:

- Recommending products based on past shopping behavior.
- Identifying fraudulent credit card transactions using anomaly detection.

20.2.3 3. Reinforcement Learning

Reinforcement learning is inspired by the way humans and animals learn through rewards and punishments.

Example:

- A mouse navigating a maze to get cheese, similar to how AI learns to play games like chess or Go.

20.3 Deep Learning

Deep learning is a specialized branch of machine learning that emulates the human brain's problem-solving capabilities.

20.3.1 Applications of Deep Learning

- Analyzing natural language (text and speech)
- Image and video processing
- Time series forecasting

20.3.2 Training Deep Learning Models

- **Data Collection & Preparation**: Raw data is labeled (e.g., bounding boxes in images for object detection).

- **Model Selection**: Choose an existing model or build one from scratch.
- **Training**: The model learns from labeled data to make predictions.
- **Evaluation & Deployment**: The model's performance is analyzed and improved before deployment.

20.3.3 Deep Learning Frameworks

Popular frameworks include:

- **TensorFlow**
- **PyTorch**
- **Keras**

These frameworks provide APIs in Python and other languages like C++ and JavaScript.

20.3.4 Pre-Trained Models

Instead of training models from scratch, developers can use pre-trained models from **model zoos** like TensorFlow Hub, PyTorch Hub, and ONNX.

21 The Model Asset eXchange

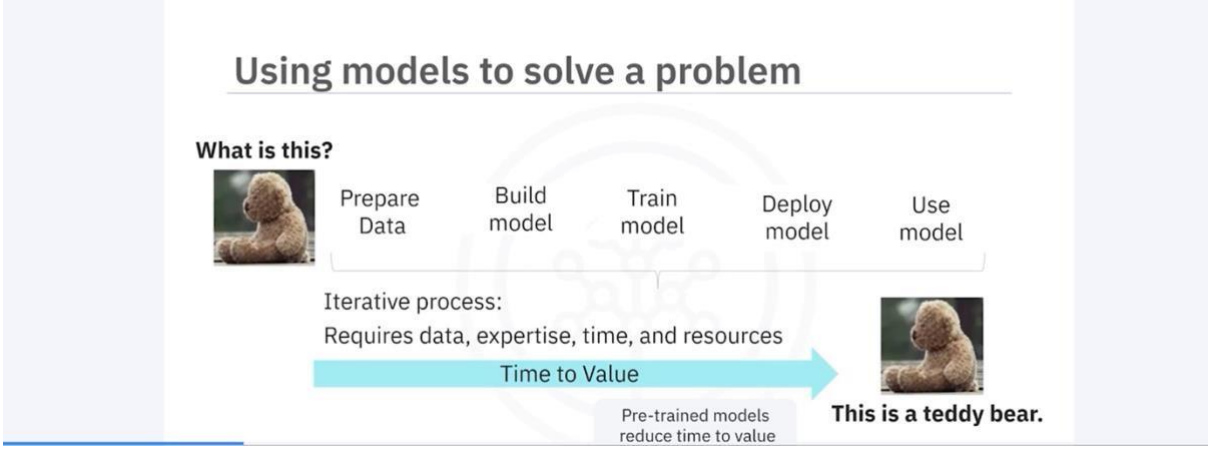
21.1 Introduction to the Model Asset eXchange

The Model Asset eXchange (MAX) on the IBM Developer platform is a free, open-source resource for deep learning models. It provides ready-to-use and customizable deep-learning microservices to help reduce the time and effort needed to train models from scratch.

21.2 Benefits of Using Pre-Trained Models

Training a deep learning model requires extensive data, labor, time, and computational resources. To accelerate the time to value, MAX offers pre-trained models that:

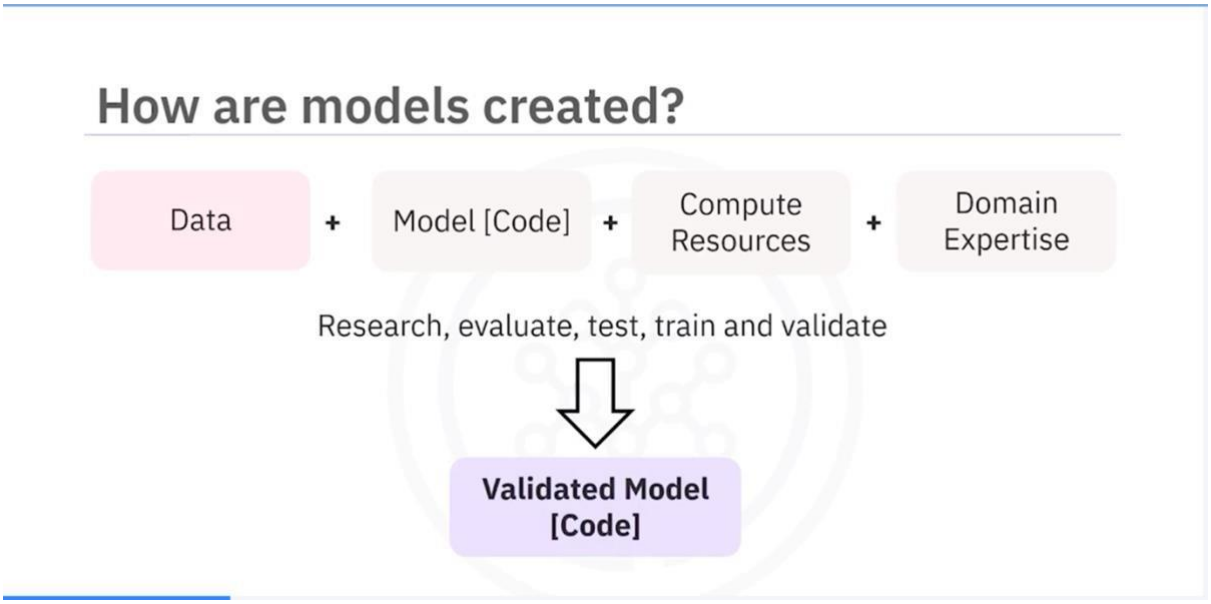
- Are ready to use immediately
- Require less time to train
- Reduce development effort



21.3 The Model Training Process

Creating a deep learning model involves multiple steps:

1. Running data through a model using computational resources and domain expertise
2. Researching and evaluating performance
3. Testing, training, and validating results
4. Producing a fully validated model



21.4 Features of the Model Asset eXchange

MAX provides a repository of pre-trained models that can be deployed in local and cloud environments. These models:

- Solve common business problems
- Are available under permissive open-source licenses
- Can be used for both personal and commercial applications without legal concerns

21.5 Model Categories Available on MAX

MAX offers deep learning models for various domains, including:

- Object detection
- Image, audio, video, and text classification
- Named entity recognition
- Image-to-text translation
- Human pose detection

21.6 Components of a Model-Serving Microservice

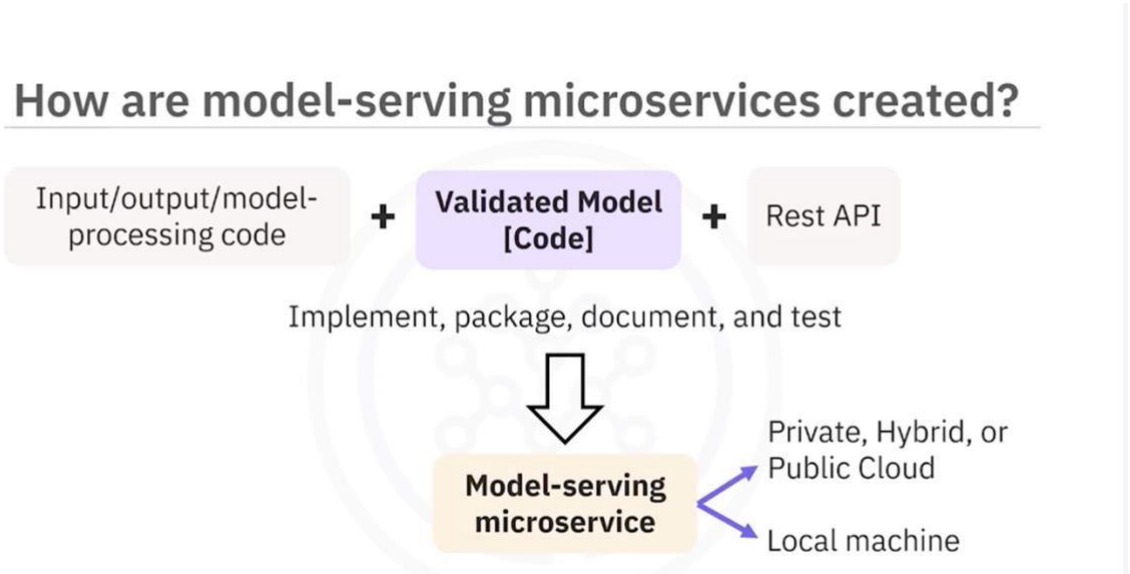
A model-serving microservice typically includes:

- A pre-trained deep-learning model
- Code for pre-processing input data before analysis
- Code for post-processing model output
- A standardized public API for application integration

21.7 Model-Serving Microservices

Model-serving microservices are created by running inputs through a validated model and exposing the results via a REST API. The development process includes:

1. Implementation
2. Packaging
3. Documentation
4. Testing
5. Deployment to local or cloud environments



21.8 Deployment and Distribution

MAX model-serving microservices are built and distributed as open-source Docker images.

21.8.1 Docker and Kubernetes

- **Docker**: A container platform for building and deploying applications
- **Kubernetes**: An open-source system for automating deployment, scaling, and management of containerized applications
- **Red Hat OpenShift**: A Kubernetes-based enterprise platform available on IBM Cloud, Google Cloud, AWS, and Microsoft Azure

21.9 Summary

- The Model Asset eXchange (MAX) is a free, open-source repository for deep learning microservices.
- Pre-trained models help reduce time to value by providing ready-to-use solutions.
- MAX model-serving microservices are built and distributed as Docker images.
- Kubernetes and Red Hat OpenShift automate deployment, scaling, and management of these microservices.

22 Getting started with the Model Asset Exchange and the Data Asset Exchange

In this lab, you will explore the Model Asset Exchange (MAX) and the Data Asset Exchange (DAX), which are two open source Data Science resources on IBM Developer.

22.1 Objective of Exercise 1:

- Find open data sets on IBM Developer.
- Explore the data sets.

22.2 Objective of Exercise 2:

- Find ready-to-use deep learning models on the Model Asset Exchange.
- Explore the deep learning model trained to detect objects in an image.

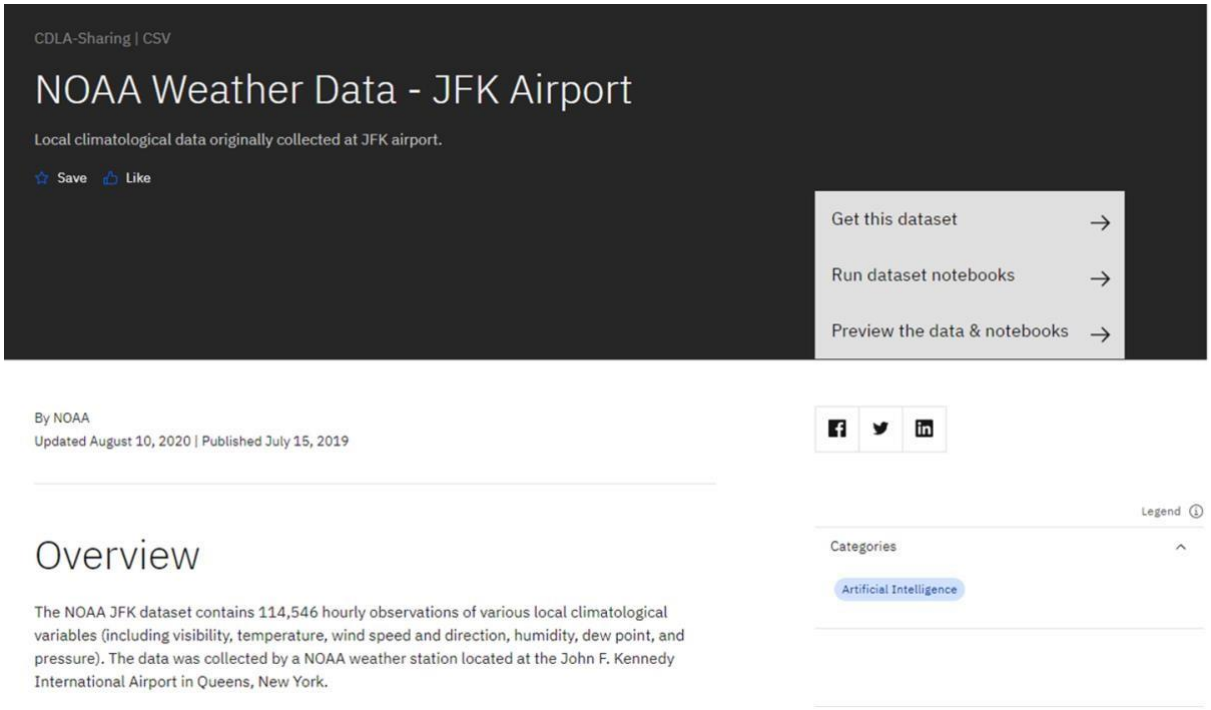
It will take you approximately 15 minutes to complete the lab. Only a web browser is required to complete the tasks.

22.3 Exercise 1: Explore deep learning datasets

The Data Asset Exchange is a curated collection of open datasets from IBM Research and third-parties that you can use to train models.

1. Open https://developer.ibm.com/exchanges/data/ in your web browser.The IBM Data Asset eXchange (DAX) home page is displayed. This is an online hub for developers and data scientists to find free and open data sets under open data licenses. These datasets can be used to train models to perform document layout analysis, natural language processing (NLP), time series analysis, and more.

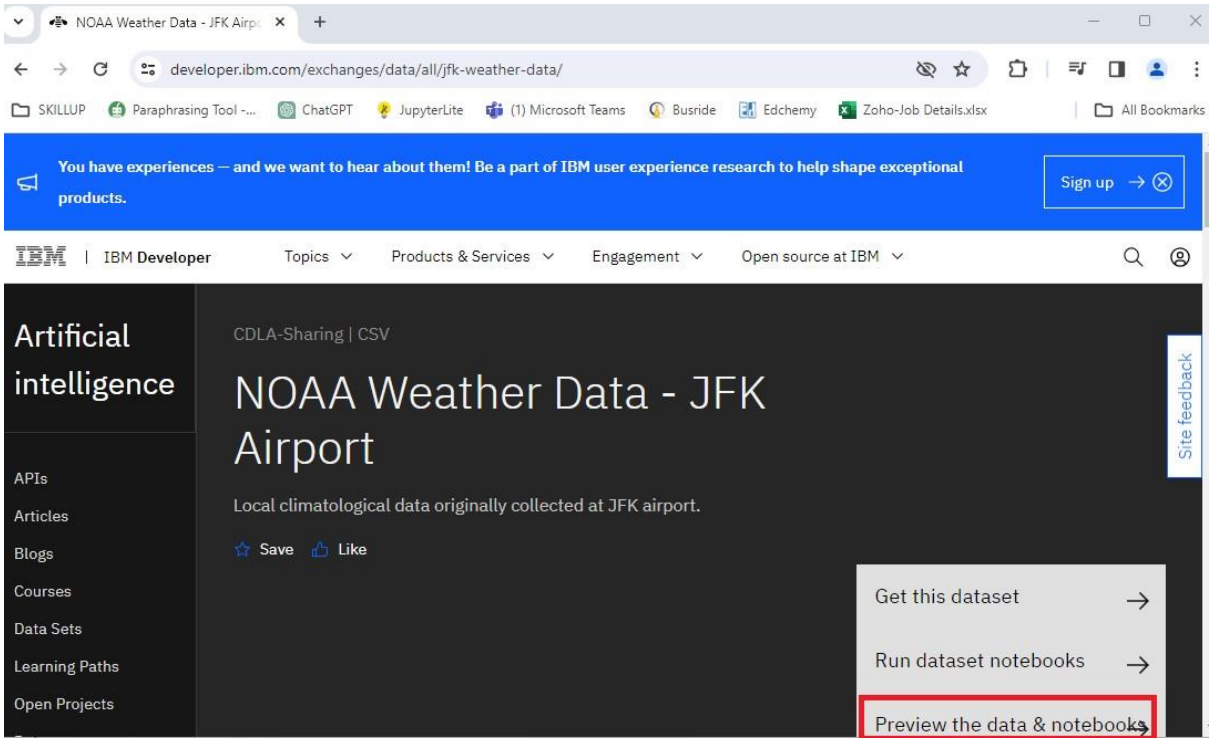
2. In this activity, you will explore **NOAA Weather Data dataset**.
3. Open the NOAA Weather Data dataset (https://developer.ibm.com/exchanges/data/all/jfk-weather-data/), which contains data from a weather station at the John F. Kennedy Airport in New York spanning eight years.
   The dataset was published under the data science friendly CDLA-Sharing license (https://cdla.io/).
   The dataset contains time-series data and can be used to predict weather trends.
   This dataset was used to train the weather forecaster model on MAX (https://developer.ibm.com/exchanges/models/all/max-weather-forecaster/).

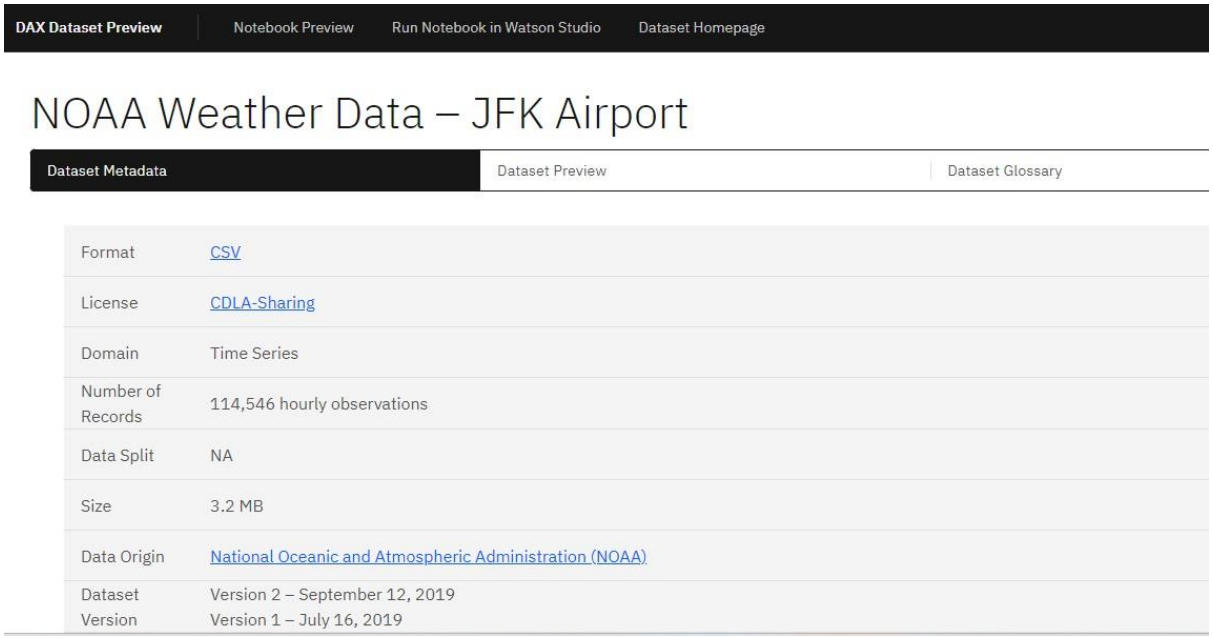


4. Inspect the dataset's metadata.
   This dataset is stored as tabular data and formatted as a comma separated value (CSV) file, which is a very popular basic data exchange format.

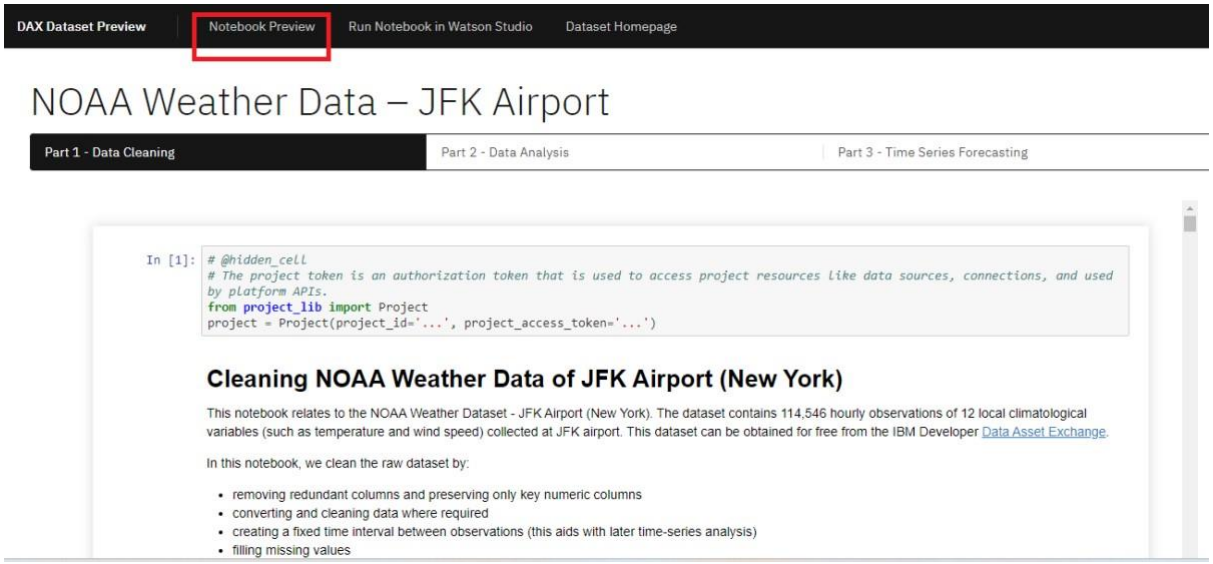
   You can download the dataset using the **Get this dataset** link. Datasets are stored as compressed archives, which you can extract using any utility that supports the targz format. If you are not familiar with this file format, take a look at this short open source tutorial https://opensource.com/article/17/7/how-unzip-targz-file.

5. Most datasets are complemented by Python notebooks that you can use to explore, pre-process, and analyze the data. The notebooks are hosted on Watson Studio, IBM's Data Science platform. Later in this course, you'll learn more about Watson Studio notebooks, how to sign up and how to run notebooks on them.
6. For now, you can preview the dataset and the notebook (or notebooks) by clicking the **Preview the data and notebooks** as shown in the screenshot below.





Now here, you click on the **Notebook Preview** option on top to view the notebook hosted with this dataset. Explore the steps followed in this notebook to clean data before performing data analysis on this dataset.



This concludes Exercise 1 of this lab, which introduced the Data Asset Exchange. You may proceed to Exercise 2.

7. [Optional] If you have already registered and are comfortable working with notebooks on Watson Studio, you can open the link using the Run Notebook in Watson Studio option and import it into a project. If you are not acquainted with IBM Watson Studio, you can skip this step. Detailed guidance on signing up and getting started with IBM Watson Studio will be provided in Module 7 of this course.

22.4 Exercise 2 - Explore deep learning models

The Model Asset Exchange is a curated repository of Open Source deep learning models for a variety of domains, such as text, image, audio, and video processing.

For more details, please visit - https://github.com/CODAIT/max-centralrepo webpage.

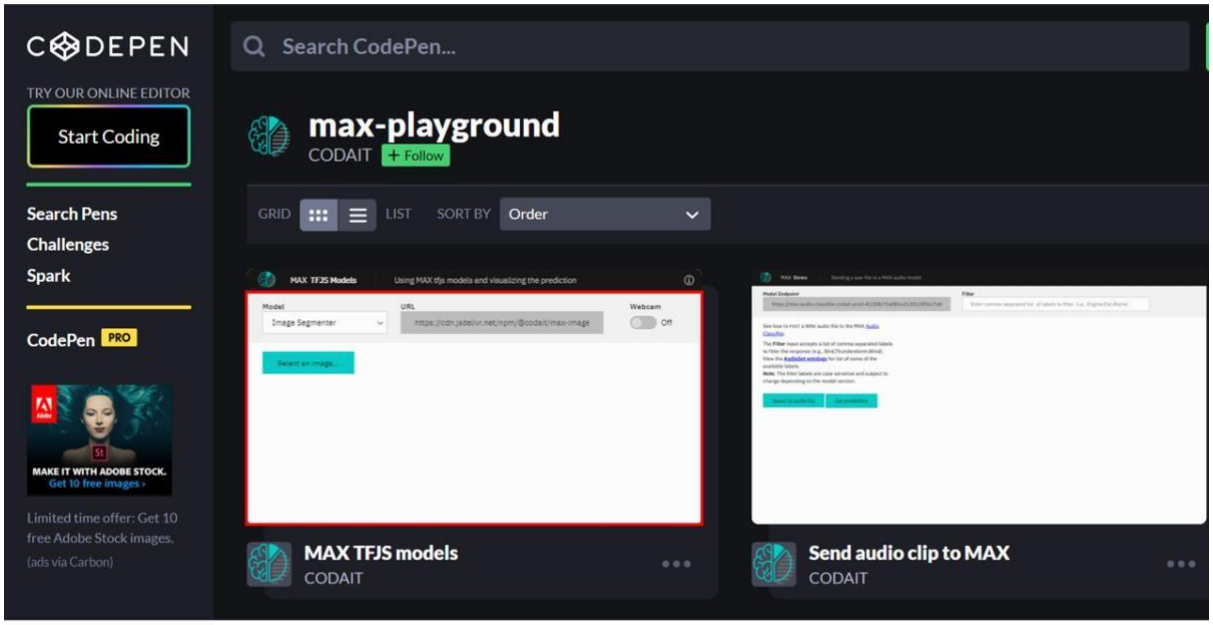
The curated list includes deployable models, which you can run as a microservice locally or in the cloud on Docker or Kubernetes, and trainable models where you can use your own data to train the models. Some of the models are already built for you to test. Let's test one of the models.

In this exercise, explore the **Object Detector** model hosted on CodePen platform. This model recognizes the objects present in an image. The model consists of a deep convolutional net base model for image feature extraction, together with additional convolutional layers specialized for the task of object detection, trained on the

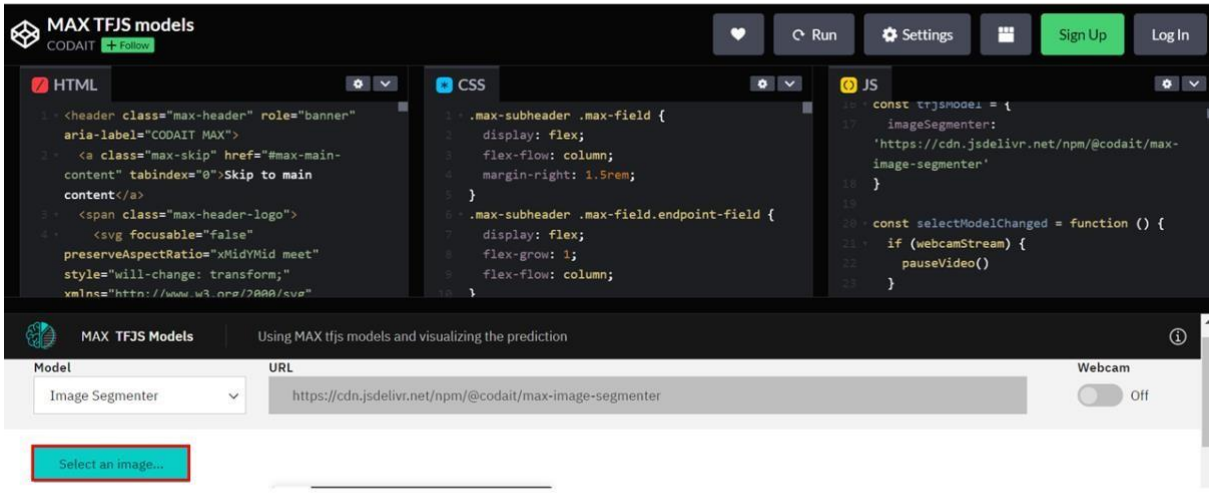
COCO data set. The input to the model is an image, and the output are extracted objects from the image, appropriately labeled.

CodePen is a social development environment. At its heart, it allows to write code in the browser and see the results of it as you build. It is a useful and liberating online code editor for developers of any skill and is particularly empowering for people learning to code.

1. Navigate to **CodePen** webpage.
2. Select **MAX TFJS models** as shown in the screenshot below. Here the **Image Segmenter**, divides an image into regions or categories that correspond to different objects or parts of objects. Every pixel in an image is allocated to one of a number of these categories.



3. Click on **Select Image** and upload an image. You may choose images with a person, dog, cat, truck, car, and so on, which are labels the model has been trained on.



Click here for all the labels the model is trained on

1. person
2. bicycle
3. car
4. motorcycle
5. airplane
6. bus
7. train
8. truck
9. boat
10. traffic light
11. fire hydrant
12. stop sign
13. parking meter
14. bench
15. bird
16. cat
17. dog
18. horse
19. sheep
20. cow
21. elephant
22. bear
23. zebra
24. giraffe
25. backpack
26. umbrella
27. handbag

28. tie
29. suitcase
30. frisbee
31. skis
32. snowboard
33. sports ball
34. kite
35. baseball bat
36. baseball glove
37. skateboard
38. surfboard
39. tennis racket
40. bottle
41. wine glass
42. cup
43. fork
44. knife
45. spoon
46. bowl
47. banana
48. apple
49. sandwich
50. orange
51. broccoli
52. carrot
53. hot dog
54. pizza
55. donut
56. cake
57. chair
58. couch
59. potted plant
60. bed
61. dining table
62. toilet
63. tv
64. laptop
65. mouse
66. remote
67. keyboard
68. cell phone
69. microwave
70. oven
71. toaster
72. sink
73. refrigerator

74. book
75. clock
76. vase
77. scissors
78. teddy bear
79. hair drier
80. toothbrush

4. Click the icon **Extract prediction** as shown below:

You can see the output of the prediction on the basis of the uploaded image.

Here the background and the dog image are separated, showing two different parts of the image.

You can also try the webcam option, which will show the real-time prediction by the toggle-on webcam option.

This concludes Exercise 2 of this lab, which introduced the Model Asset Exchange (MAX).

23 Module 3 Summary

- Python offers a diverse library ecosystem for data science, covering scientific computing (Pandas, NumPy), visualization (Matplotlib, Seaborn), and high-level machine learning (Scikit-learn). These libraries offer tools for data manipulation, mathematical operations, and simplified machine learning model development.
- Application Programming Interfaces (APIs) facilitate communication between software components. REST APIs, specifically, facilitate internet communication and access resources like storage. Key API terms include client (user or code accessing it), resource (service or data), and endpoint (API's URL).
- Machine learning models analyze data and identify patterns to make predictions and automate complex tasks—the three fundamental types of machine learning are supervised, unsupervised, and reinforcement learning. Supervised learning includes regression and classification models for predictive modeling and pattern recognition. Deep learning, an advanced subset of machine learning, mimics the brain's processing, enabling intricate problem-solving in various domains.
- The Community Data License Agreement (CDLA) facilitates open data sharing by providing clear licensing terms for distribution and use, and the IBM Data Asset eXchange (DAX) site contains high-quality open data sets.
- The Model Asset eXchange (MAX) provides a wealth of pre-trained deep learning models, empowering developers with readily deployable solutions for various business challenges.

Module 4

24 Introduction to Jupyter Notebooks

24.1 What is a Jupyter Notebook?

Jupyter Notebooks originated as **iPython**, which was initially developed for Python programming. As it expanded to support multiple languages, it was renamed **Jupyter**, representing Julia, Python, and R. Today, it supports many more languages.

A Jupyter Notebook is a **browser-based application** that allows users to create and share documents containing:

- Code

- Equations
- Visualizations
- Narrative text
- Links

It is similar to a scientist's lab notebook, where all steps of an experiment and its results are recorded for reproducibility. Likewise, Jupyter Notebooks help **Data Scientists** document their data experiments and results for reuse by others.

24.2 Features of Jupyter Notebooks

- Allows combining **descriptive text, code blocks, and code output** in a single file.
- Generates outputs such as **plots and tables** within the notebook file.
- Supports exporting notebooks to **PDF or HTML** formats for easy sharing.

24.3 Introduction to JupyterLab

JupyterLab is an **enhanced, browser-based interface** that extends the functionalities of Jupyter Notebooks. It allows users to:

- Access multiple **Jupyter Notebook files, code, and data files**.
- Work with **multiple notebooks, text editors, terminals, and custom components** in an integrated manner.
- Use a variety of **file formats**, including CSV, JSON, PDF, and Vega.
- Benefit from an **open-source** environment.

24.4 Cloud-Based Usage of Jupyter Notebooks

Jupyter Notebooks can be used with cloud-based services like **IBM Cloud and Google Colab**, which offer:

- No need for local installation.
- Access to the **Jupyter Notebook environment** directly in the cloud.
- Import and export capabilities using the **standard IPython Notebook file format**.
- Support for **Python and other programming languages**.

24.5 Installing Jupyter Notebooks

Jupyter Notebooks can be installed in different ways:

- **Via Command Line**: Using the command:

-

  pip install jupyter
- **Using Anaconda**: Downloading Jupyter from the **Anaconda Platform** at Anaconda.com. Anaconda is a popular distribution that includes both **Jupyter and JupyterLab**.

24.6 Summary

- **Jupyter Notebooks** are widely used in **Data Science** for recording experiments and projects.
- **JupyterLab** extends Jupyter Notebook functionalities, supporting multiple file types and programming languages.
- Jupyter Notebooks can be installed locally or used via **cloud-based services** like Google Colab and IBM Cloud.
- This course provides access to a **pre-hosted JupyterLab environment**, so no installation is required.

25 Getting Started with Jupyter

25.1 Working with a Jupyter Notebook

Jupyter notebooks allow users to create, edit, and execute code in an interactive environment. This section covers the basic operations of running, inserting, and deleting cells.

25.1.1 Running a Cell

- Click the **Run** button on the toolbar.
- Alternatively, click **Run** in the main menu bar and select **Run Selected Cells**.
- Use the shortcut **Shift + Enter** to run the highlighted cell.
- To run all cells at once, click **Run All Cells**.

25.1.2 Inserting and Deleting Cells

- To **insert** a new cell, click the **plus (+) symbol** on the toolbar.
- To **delete** a cell:
  - Highlight the cell, then click **Edit > Delete Cells**.
  - Alternatively, press **D twice** on the highlighted cell.
- Cells can be moved up or down as needed.

25.2 Working with Multiple Notebooks

Jupyter allows multiple notebooks to be open simultaneously.

25.2.1 Opening a New Notebook

- Click the **plus (+) button** on the toolbar and select the file you want to open.
- Alternatively, click **File > Open a New Launcher** or **File > Open a New Notebook**.
- You can arrange notebooks side by side for better workflow.

25.2.2 Executing Code in Multiple Notebooks

- Assign variables and perform calculations across multiple notebooks.
- Example: Assign variable_one = 1 and variable_two = 2, then print their sum.

25.3 Presenting Results in Jupyter

Jupyter supports presenting results directly within the notebook.

25.3.1 Using Markdown for Presentation

- Markdown cells allow adding titles and text descriptions.
- To add Markdown:
  - Click **Code** and select **Markdown**.
  - Use Markdown syntax for better formatting.

25.3.2 Creating Slides in Jupyter

- Convert code cells, text, and visualizations into slides.
- Slides allow structured project presentations with code execution outputs.

25.4 Shutting Down Jupyter Notebook Sessions

Closing notebooks properly helps release memory and system resources.

25.4.1 Shutting Down a Notebook

- Click the **Stop icon** on the sidebar (second icon from the top).
- Choose to **terminate all sessions** at once or shut them down individually.
- After shutting down, the message **"no kernel"** appears at the top right, indicating the notebook is inactive.

25.5 Summary

- Learned to run, delete, and insert a code cell.
- Opened and worked with multiple notebooks simultaneously.
- Used Markdown and code cells for presentations.
- Properly shut down notebook sessions after completing the work.

26 Hands-on Lab: Getting Started with Jupyter Notebooks

After completing this lab, you will be able to:

Get an overview of code and markdown cells

- Execute an existing code cell

•

- Insert and delete a code cell
- Write comments in Python
- Create and use Markdown cells

27 Jupyter Kernels

27.1 Introduction to Jupyter Kernels

A notebook kernel is a computational engine that executes the code contained in a Notebook file. Jupyter supports kernels for multiple programming languages, making it a versatile tool for Data Science.

27.2 How Jupyter Kernels Work

- When a Notebook document opens, the related kernel launches automatically.
- Upon execution, the kernel processes the code and returns the results.
- Some environments may require additional language installations to use different kernels.

27.3 Pre-Installed Kernels in Skills Network Lab

In the Skills Network lab environment, several kernels come pre-installed:

27.3.1 Python Kernel

- The default kernel in Jupyter.
- Allows execution of Python code within notebook cells.
- Displays output upon execution.

27.3.2 Other Available Kernels

- Apache
- Julia
- R
- Swift
- Users can select the kernel at launch or switch it using the drop-down menu in the top right corner.

27.4 Installing Kernels on a Local Machine

- If using Jupyter locally, additional kernels must be installed manually.
- Installation is done through the command line interface (CLI) based on the language requirement.

27.5 Summary

- The kernel functions as a computational engine, executing code within Jupyter Notebooks.

- Jupyter supports multiple languages, enabling flexibility in Data Science projects.
- Users can switch between different kernels as needed, either in cloud environments or locally.

28 Jupyter Architecture

28.1 Overview

Jupyter architecture follows a two-process model consisting of a **kernel** and a **client**. After completing this section, you will be able to:

- Describe the basic Jupyter architecture.
- Explain Jupyter architecture for file format conversion.

28.2 Jupyter's Two-Process Model

Jupyter operates with two main components:

28.2.1 Client

- The **client** serves as the interface that allows users to send code to the kernel.
- In a Jupyter Notebook, the browser acts as the client.

28.2.2 Kernel

- The **kernel** is responsible for executing the code and returning the results to the client for display.
- It processes the code within notebook cells and provides outputs accordingly.

28.3 Jupyter Notebooks Structure

Jupyter Notebooks store various elements, including:

- **Code**
- **Metadata**
- **Contents**
- **Outputs**

When saving a notebook:

- The browser sends the notebook to the **Notebook Server**.
- The **Notebook Server** saves the file on disk as a JSON file with a .ipynb extension (pronounced as **dot i PI NB**).

The **Notebook Server** is responsible for both saving and loading notebooks.

•

28.4 Jupyter Architecture for File Conversion

Jupyter supports converting notebook files into other formats using the **NB Convert Tool**. The conversion process includes the following steps:

28.4.1 1. Preprocessing

- The notebook file is first modified by a **preprocessor** to prepare it for export.

28.4.2 2. Exporting

- The **exporter** converts the notebook into the target file format, such as **HTML**.

28.4.3 3. Postprocessing

- A **postprocessor** works on the exported file to generate the final output.
- After conversion, providing the URL of the file enables its display as an **HTML file**.

28.5 Summary

- Jupyter architecture follows a **two-process model** with a **kernel** and a **client**.
- The **Notebook Server** is responsible for saving and loading notebooks.
- The **kernel** executes code cells within the notebook.
- The **NB Convert Tool** is used to convert Jupyter notebooks into other formats.
- The conversion process includes **preprocessing, exporting, and postprocessing** to generate the final output.

29 Hands-on Lab: Working with Files in Jupyter Notebooks

After completing this lab, you will be able to:

- Explore different Run Options
- Rename your notebook
- Shut down your notebook
- Save and download the notebook
- Upload the downloaded notebook
- Work with multiple notebooks

30 Additional Anaconda Jupyter Environments

30.1 Overview

After watching this video, you will be able to:

- Describe Anaconda and its data science features.

- Explain Anaconda Jupyter environments.
- Identify tools in Anaconda Jupyter environments.

30.2 Computational Notebooks

Computational notebooks combine code, computational output, explanatory text, and multimedia resources into a single document. Jupyter Notebook is a popular computational notebook because it supports multiple programming languages.

30.2.1 Jupyter Environments

- **JupyterLab**: An open-source, web-based application based on Jupyter Notebook. It includes features such as interactive visualizations, text, and equations.
- **VS Code**: A widely used code editor that supports multiple languages, syntax highlighting, and auto-indentation.

30.3 Anaconda: A Data Science Platform

Anaconda is a free and open-source distribution for Python and R, the top languages used in data science and machine learning.

30.3.1 Features of Anaconda

- Includes over 1,500 libraries.
- Free to install with community support.
- Comes with **Anaconda Navigator**, a graphical user interface that allows users to install new packages without using the command line.

30.3.2 Launching JupyterLab in Anaconda Navigator

1. Open **Anaconda Navigator**.
2. Locate the **JupyterLab** box.
3. Click **Launch**. If the button is missing, click **Install** first, then **Launch**.

30.3.3 Opening Jupyter Notebook

1. In the search bar, type Jupyter Notebook (anaconda3) and press **Enter**.
2. The **JupyterLab dashboard** opens in the browser on localhost.
3. To create a new notebook:
    - Click **New** and select **Python 3**. o Rename the notebook by clicking **Untitled**, typing a new name, and clicking **Rename**.

30.4 Jupyter Notebook Cell Types

- **Code Cell**: Contains code to be executed in the kernel and displays its output.

- **Markdown Cell**: Contains rich text and displays formatted content when executed.

30.5 Downloading a Jupyter Notebook

1. Click **File**.
2. Select **Download As**.
3. Choose the desired file format.

30.6 Using VS Code for Jupyter Notebooks

VS Code is a free, open-source editor used for debugging and task execution.

30.6.1 Installing VS Code

1. Visit code.visualstudio.com.
2. Download the version for your device.
3. Follow the installation instructions.

30.6.2 Opening VS Code with Anaconda Navigator

1. Open **Anaconda Navigator**.
2. Find the **VS Code** application.
3. Click **Launch**.

30.6.3 Configuring VS Code for Jupyter Notebooks

1. Open VS Code.
2. Click **Extensions** or use **Ctrl + Shift + X**.
3. Search for **Python** and install relevant extensions.
4. Create a new Jupyter Notebook:
   - Click **File → New File**.
   - Select **Jupyter Notebook**.
   - Write and execute code using the **Run** icon.

30.6.4 Saving Jupyter Notebooks in VS Code

1. Click **File**.
2. Select **Save**.

30.7 Summary

- Jupyter is a powerful computational notebook that supports multiple programming languages.
- Anaconda Navigator provides an easy way to launch JupyterLab and VS Code.
  VS Code can be used as an alternative to JupyterLab but requires additional configuration.

-
- Jupyter environments can be downloaded separately from Anaconda Navigator, but they may not be configured properly.

31 Additional Cloud-Based Jupyter Environments

31.1 Overview

After reading this, you will be able to:

- Describe cloud-based Jupyter environments and their data science features.
- Navigate cloud-based Jupyter environments.
- Identify tools in cloud-based environments.

31.2 Computational Notebooks

Computational notebooks combine:

- Code
- Computational output
- Explanatory text
- Multimedia resources

Jupyter Notebook is a popular computational notebook because it supports multiple programming languages.

31.3 Cloud-Based Jupyter Environments

Popular cloud-based environments for creating and modifying Jupyter notebooks include:

- **JupyterLite**
- **Google Colaboratory**

31.3.1 JupyterLite

JupyterLite is a lightweight tool built from JupyterLab components that runs entirely in the browser.

- It does not require a dedicated Jupyter server.
- It only needs a web server, allowing it to be deployed as a static website.
- It supports interactive graphics and visualizations using libraries like Altair, Plotly, and ipywidgets.
- As a JupyterLab distribution, it includes the latest improvements and features.

31.3.1.1 Launching JupyterLite

1. Open a browser and type jupyter.org/try-jupyter/lab in the URL field.
2. Press **Enter**.
3. Click **Python (Pyodide)**.

31.3.1.2 JupyterLite Kernel

- The **kernel** in JupyterLite determines how code is executed.
- The default kernel for JupyterLite is **Pyolite**, a Python kernel based on Pyodide.
- Pyolite runs in the background to enable faster execution of intensive computations.
- Other kernels can also be used with JupyterLite.

31.3.2 Google Colaboratory

Google Colaboratory (Google Colab) is a free Jupyter notebook environment that runs entirely in the cloud.

- Google Colab notebooks execute in a browser.
- Projects are stored on **Google Drive** and **GitHub**.
- It allows uploading and sharing notebooks without requiring setup or installation.
- Supports cloning projects from **GitHub** and running them in Google Colab.
- Pre-installed libraries include **scikit-learn** and **matplotlib**.
- Enables rapid development of data science applications **without extensive setup**.

31.3.2.1 Opening Google Colab

1. Open **Google Drive**.
2. Click **New**.
3. From the Google Drive menu, select **More → Google Colaboratory**.

31.3.2.2 Using Google Colab

- Write code in the **code section**.
- Click the **Run** icon to execute the code.
- To add more cells:
  - Click **+Code** to add a new code cell.
  - Click **+Text** to add a text cell.
  - Text cells can be used for rich text or set as **Markdown** cells.

31.4 Summary

- Jupyter is a popular computational notebook tool supporting multiple programming languages.
- The **Anaconda Navigator** GUI can launch multiple applications.
  Additional open-source Jupyter environments include **JupyterLab, JupyterLite, VS Code, and Google Colaboratory**.

-

- JupyterLite is a **browser-based** tool requiring no server setup.

32 Jupyter Notebooks on the Internet

There are thousands of interesting Jupyter Notebooks available on the internet for learning purposes. One of the best sources is:

- Jupyter Wiki on GitHub

32.1 Accessing Jupyter Notebooks

You can download these notebooks to your local computer or import them into a cloud-based notebook tool to rerun, modify, and apply the concepts explained in the notebook.

32.1.1 Rendered vs. Unrendered Notebooks

- Many Jupyter Notebooks are shared in a **rendered view**, allowing you to explore them as if they were running locally.
- Some are shared only as a link to a Jupyter file with the **.ipynb** extension.
- In such cases, you can paste the URL of the notebook into **NB-Viewer** to render it: NB-Viewer

32.2 Noteworthy Jupyter Notebooks for Data Science

The list of Jupyter Notebooks provides a vast collection of learning materials. Below are some recommended notebooks for different data science tasks:

32.2.1 1. Exploratory Data Analysis (EDA)

- **Notebook:** Exploratory Data Analysis

32.2.2 2. Data Integration and Cleansing

- **Notebook:** Data Cleaning with Pandas

32.2.3 3. Clustering

- **Notebook:** K-Means Clustering

32.2.4 4. Iris Dataset Analysis

- **Notebook:** Iris Dataset Exploratory Data Analysis

These notebooks offer valuable insights into data science concepts and hands-on examples for further exploration.

33 Module 4 Summary

- Jupyter Notebooks are used in Data Science for recording experiments and projects.
- Jupyter Lab is compatible with many files and Data Science languages.
- There are different ways to install and use Jupyter Notebooks.
- How to run, delete, and insert a code cell in Jupyter Notebooks.
- How to run multiple notebooks at the same time.
- How to present a notebook using a combination of Markdown and code cells.
- How to shut down your notebook sessions after you have completed your work on them.
- Jupyter implements a two-process model with a kernel and a client.
- The notebook server is responsible for saving and loading the notebooks.
- The kernel executes the cells of code contained in the Notebook.
- The Jupyter architecture uses the NB convert tool to convert files to other formats.
- Jupyter implements a two-process model with a kernel and a client.
- The Notebook server is responsible for saving and loading the notebooks.
- The Jupyter architecture uses the NB convert tool to convert files to other formats.
- The Anaconda Navigator GUI can launch multiple applications on a local device.
- Jupyter environments in the Anaconda Navigator include JupyterLab and VS Code.
- You can download Jupyter environments separately from the Anaconda Navigator, but they may not be configured properly.
- The Anaconda Navigator GUI can launch multiple applications.
- Additional open-source Jupyter environments include JupyterLab, JupyterLite, VS Code, and Google Colaboratory.
- JupyterLite is a browser-based tool.

Module 5

34 Introduction to R and RStudio

34.1 What is R?

R is a statistical programming language widely used for:

- Data processing and manipulation
- Statistical inference
- Data analysis
- Machine learning algorithms

34.1.1 Usage of R

Based on a 2017 analysis, R is primarily used in:

- Academia
  Healthcare
- Government

•

34.1.2 Data Import Capabilities

R supports importing data from various sources, including:

- Flat files
- Databases
- Web sources
- Statistical software (SPSS, STATA)

34.2 Why Choose R?

R is a preferred language for some data scientists due to:

- Easy-to-use functions
- Powerful data visualization capabilities
- Built-in packages for data analysis without requiring additional installations

34.3 RStudio: Integrated Development Environment for R

RStudio is a popular IDE for developing and running R code. It enhances productivity and provides an efficient interface for coding.

34.3.1 Features of RStudio

- **Syntax-highlighting editor**: Supports direct code execution and keeps records of work
- **Console**: Allows typing and executing R commands
- **Workspace and History tab**: Displays created R objects and a history of executed commands
- **Files tab**: Shows files in the working directory
- **Plots tab**: Displays the history of created plots and allows exporting to PDF or image files
- **Packages tab**: Lists available external R packages on the local system
- **Help tab**: Provides resources for R, RStudio support, and package documentation

34.4 Popular R Libraries for Data Science

For those using R in Data Science, here are some essential libraries:

- **dplyr**: Data manipulation
- **stringr**: String manipulation
- **ggplot**: Data visualization
- **caret**: Machine learning

34.5 Virtual Lab for RStudio

To help learners practice without setup hassle, an **RStudio virtual environment** is provided as part of the Skills Network Labs. This environment allows hands-on learning without requiring account creation, downloads, or installations.

34.6 Summary

- R is a powerful statistical programming language used in academia, healthcare, and government.
- It supports importing data from multiple sources and offers built-in data analysis capabilities.
- RStudio enhances productivity by providing a structured environment for coding in R.
- Essential R libraries like dplyr, ggplot, and caret are widely used in Data Science.
- A virtual RStudio environment is available to facilitate hands-on learning without additional setup.

35 Optional Reading: Download & Install R and RStudio 36 R Basics with RStudio

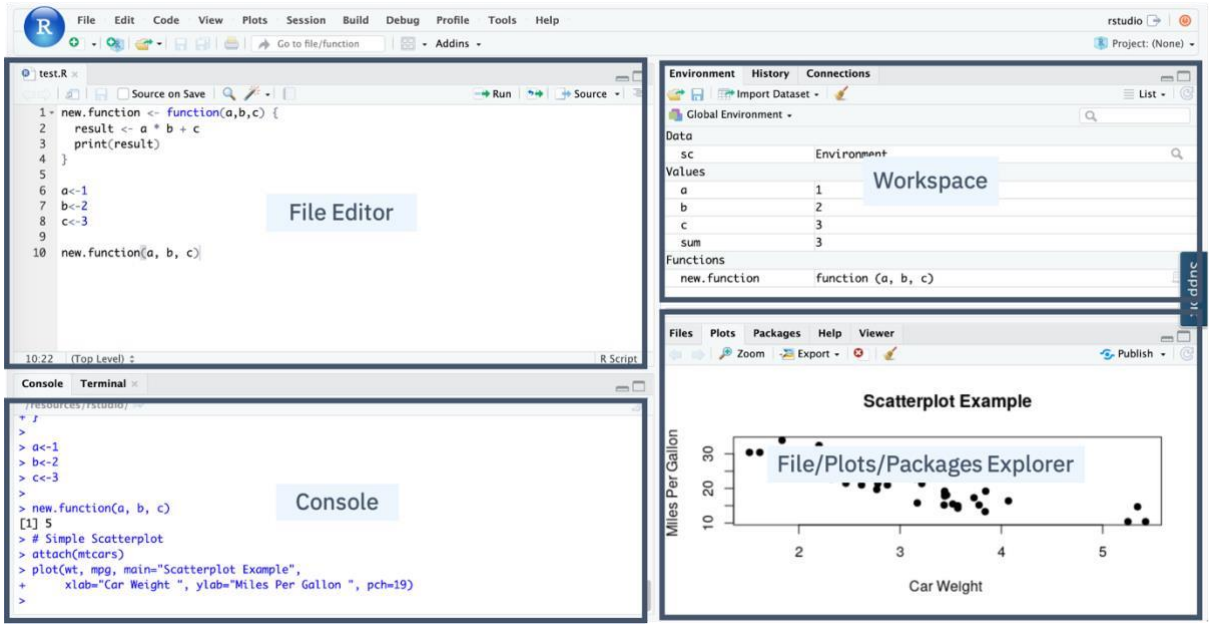
After completing this lab, you will:

- Get familiar with RStudio
- Write your first R code snippet in RStudio

36.1 RStudio main UI

In this lab, you will be introduced to RStudio, the most popular and powerful IDE for developing R projects.

The main UI of RStudio is shown here:



1. In the Console panel, you can quickly try some R commands and see the results immediately.
2. In the File Editor panel, you can write your R code or other text files with the help of syntax highlighting and auto-completion.
3. In the Workspace panel, you can review and manage the created objects.
4. In the File/Plots/Package Explorer panel, you can manage your files and other assets, such as plots or packages.

37 Plotting in RStudio

37.1 Objectives

After watching this video, you will be able to:

- List the R data visualization packages.
- Use the inbuilt R plot function.
- Use the R ggplot library to add functions and arguments to the plot.
- Add titles and names to the plot.

37.2 Importance of Data Visualization

With the influx of data, one of your key responsibilities as a data scientist is to generate insights using visualizations. R provides various packages for data visualization, which can be selected based on specific requirements.

37.3 R Data Visualization Packages

To install these packages in your R environment, use the install.packages("package_name") command. Some popular R packages for data visualization include:

37.3.1 ggplot

- Used for creating data visualizations such as histograms, bar charts, scatterplots, etc.
- Allows adding multiple layers and components to a single visualization.

37.3.2 Plotly

- Used for web-based interactive visualizations.
- Plots can be displayed in browsers or saved as individual HTML files.

37.3.3 Lattice

- Used for handling complex, multi-variable datasets.
- A high-level visualization library that requires minimal customization.

37.3.4 Leaflet

- Used for creating interactive plots, especially for mapping and geographic visualizations.

37.4 Plotting with the Inbuilt R Plot Function

R has built-in functions to create plots and visualizations. For example, the plot() function generates a scatterplot of values versus the index.

37.4.1 Enhancing Plots

- **Adding Lines:** Use the type argument to add a line to the plot.
- **Adding Titles:** Use the title() function to make the visualization more readable.

37.5 Plotting with ggplot

The ggplot library in R allows the creation of detailed and informative visualizations by adding layers and components to plots using various functions and arguments.

37.5.1 Creating a Scatter Plot with ggplot

To create a scatter plot using the inbuilt mtcars dataset:

1. Load the ggplot library using library(ggplot2).
2. Use the ggplot() function on the mtcars dataset.
3. Set the X-axis as **miles per gallon (mpg)** and the Y-axis as **weight (wt)**.
4. Add the geom_point() function to specify a scatter plot. Without this, an empty plot will be generated.
5. The output is a well-structured scatter plot.

37.5.2 Enhancing ggplot Visualizations

- **Adding Titles:** Use the ggtitle() argument to add a meaningful title.
- **Renaming Axes:** Use the labs() argument to specify appropriate axis names.
- The final result will be a well-labeled graph with clear and informative titles.

37.6 Extending ggplot with GGally

In the lab, you will use ggplot along with the GGally extension library.

- GGally extends ggplot by adding multiple functions that simplify combining geometric objects with transformed data.

37.7 Summary

- R provides various data visualization packages, including ggplot, Plotly, Lattice, and Leaflet.
- The inbuilt plot() function helps create basic plots in R.
- ggplot allows detailed visualizations by adding layers and components.
- You can modify ggplot outputs by adding titles (ggtitle) and renaming axes (labs).
- GGally extends ggplot by simplifying complex data visualizations.

38 Getting started with RStudio and Installing packages

In this lab, you will learn how to get started with RStudio and install packages.

39 Creating Data Visualizations using ggplot

In this lab, you will learn how to create some basic visualizations in R.

40 Plotting with RStudio

This lab introduces you to advanced plotting with ggplot.

41 Introduction to Git and GitHub

41.1 Overview

Git and GitHub are popular tools among developers and data scientists for version control and collaboration. Before diving into Git and GitHub, it's essential to understand version control.

41.2 What is Version Control?

A version control system helps track changes to documents, making it easier to:

- Recover older versions if mistakes are made.
- Collaborate effectively with others.

41.2.1 Example of Version Control

Imagine you have a shopping list and want your roommates to confirm and add items. Without version control, merging everyone's changes can be chaotic. With version control, you can track all modifications and ensure a clear final list.

41.3 Understanding Git

Git is a free and open-source software distributed under the GNU General Public License. It is a **distributed version control system**, meaning users worldwide can have a local copy of a project and sync changes to a remote server.

41.3.1 Why Use Git?

- Unlike other version control systems, Git's distributed nature allows multiple users to work independently and merge changes seamlessly.
- Version control systems are used not only for coding but also for managing images, documents, and various file types.

41.4 Git vs. GitHub

- **Git** is a command-line tool for version control.
- **GitHub** is a web-based hosting service for Git repositories.
- Other alternatives to GitHub include **GitLab, BitBucket, and Beanstalk**.

41.5 Key Git and GitHub Terminology

- **SSH Protocol**: A secure method for remote login from one computer to another.
- **Repository (Repo)**: A project folder set up for version control.
- **Fork**: A copy of a repository.
- **Pull Request**: A request to review and approve changes before they are merged.
- **Working Directory**: The files and subdirectories on your computer linked to a Git repository.

41.6 Essential Git Commands

41.6.1 Initializing a Repository

- `git init` – Creates a new Git repository.
- `git clone [URL]` – Clones an existing repository.

41.6.2 Staging and Committing Changes

- `git add [file]` – Moves changes from the working directory to the staging area.
- `git status` – Displays the current state of the working directory and staging area.
- `git commit -m "message"` – Commits staged changes to the project.

41.6.3 Undoing Changes

- `git reset [file]` – Undoes changes made to files in the working directory.

41.6.4 Viewing and Managing Changes

- `git log` – Displays the history of commits.
- `git branch` – Creates an isolated branch within the repository.
- `git checkout [branch]` – Switches between branches.
- `git merge [branch]` – Merges changes from one branch into another.

41.7 Learning Git and GitHub

To get started with Git and GitHub, visit **try.github.io** for cheat sheets and tutorials. In the following modules, we will cover setting up your local environment and starting your first project.

42 Introduction to GitHub

42.1 Purpose of Source Repositories

After watching this video, you will be able to:

- Describe the purpose of source repositories.
- Explain how GitHub satisfies the needs of a source repository.

42.2 History of Git Development

Linux development in the early 2000s was managed under a free-to-use system known as BitKeeper. In 2005, BitKeeper transitioned to a for-fee system, which posed challenges for Linux developers. Linus Torvalds led a team to develop a replacement version-control system, resulting in the creation of Git.

42.2.1 Key Characteristics of Git

Git was designed based on several crucial characteristics:

- **Strong support for non-linear development** – At the time, Linux patches arrived at a rate of 6.7 patches per second.
- **Distributed development** – Each developer has a local copy of the full development history.
- **Compatibility with existing systems and protocols** – This was necessary to accommodate the diverse Linux community.
- **Efficient handling of large projects** – Git ensures scalability for massive codebases.
- **Cryptographic authentication of history** – Ensures that all distributed systems maintain identical code updates.
- **Pluggable merge strategies** – Supports complex integration decisions requiring explicit strategies.

42.3 Git Repository Model

42.3.1 Features of Git

Git is a **distributed version-control system** designed to:

- Track source code during development.
- Coordinate collaboration among programmers.
- Support non-linear workflows.
- Maintain a history of changes.

- Enable distributed development, allowing each developer to act as a hub.

42.3.2 Central vs. Distributed Version Control

- **Centralized Version Control:** Developers check out code from a central system and commit changes back to it.
- **Distributed Version Control:** Each developer maintains a full local copy of the repository, enabling efficient collaboration and parallel development.

42.3.3 Git Workflow

- A main branch corresponds to the deployable code.
- Teams can integrate changes continuously for releases.
- Developers can work on separate branches in between releases.
- Centralized administration is possible with access-level controls.

42.3.4 Using Git

Git can be used locally via:

- **GitHub Desktop client**
- **GitHub web interface**

42.4 Introduction to GitHub

GitHub is an **online hosting service for Git repositories**, offering free, professional, and enterprise accounts. It is owned by Microsoft and, as of August 2019, hosted over **100 million repositories**.

42.4.1 GitHub Features

- Provides a central location for collaboration.
- Supports agile development methodologies.
- Enables developers to track and maintain version control.

42.5 Understanding Repositories

A **repository (repo)** is a data structure for storing documents, including application source code. It supports version control, allowing contributors to:

- Track changes.
- Maintain different versions of the codebase.

42.6 GitLab: An Alternative DevOps Platform

GitLab is a **complete DevOps platform** that provides access to Git repositories controlled by source code management.

42.6.1 GitLab Features

With GitLab, developers can:

- Collaborate by reviewing code, making comments, and improving each other's work.
- Work from their own local copy of the code.
- Branch and merge code when required.
- Streamline testing and delivery with built-in **Continuous Integration (CI)** and **Continuous Delivery (CD)**.

42.7 Summary

- **GitHub** is an online hosting service for Git repositories.
- **Repositories** store documents, including application source code, and support version control.
- **Git** is a distributed version-control system that enables collaboration, tracking, and non-linear workflows.
- **GitLab** offers an integrated DevOps platform for managing source code and deployment.

These concepts form the foundation of modern version control and collaborative software development.

43 GitHub Repositories

43.1 Signing Up for a GitHub Account

Creating a free, personal account on GitHub is quick and easy:

1. Visit GitHub's website.
2. Choose a username, enter your email address, and create a password.
3. Click **Sign up for GitHub**.
4. Complete a short verification test to confirm you're a person.
5. Click **Verify** and solve the puzzle presented.
6. Select a free, personal account (default option).
7. Answer GitHub's optional questions about work, programming experience, and interests (or skip them).
8. Confirm your email to activate your account.

43.2 Getting Started with GitHub

After signing up, GitHub provides several starting points:

- **Create a Repository** – A data structure for storing documents and tracking version control.
- **Create an Organization** – A collection of user accounts managing repositories.
- **Take the Introduction to GitHub Course** – A learning resource for new users.

Alternatively, you can skip these steps and start working immediately. GitHub also offers a guide to help users navigate its features effectively.

43.3 Understanding Repositories

A **repository** is the core of a Git-based project, storing all code and related files. It can contain:

- **README file** – Describes the project's purpose.
- **License** – Defines how others can use your code.
- **Visibility Settings** – Repositories can be **private** (restricted access) or **public** (accessible to everyone).

43.3.1 Repository Tabs

When you create a repository, you'll see several tabs:

43.3.1.1 Code

- Contains all source files.
- If a README or license file was created, it appears here.

43.3.1.2 Issues

- A tool for tracking and planning work on your project.
- Lists all open issues.

43.3.1.3 Pull Requests

- Allows collaboration with others.
- Defines changes that are committed and ready for review before merging into the main branch.

43.3.1.4 Projects

- Provides tools for managing, sorting, and planning various projects.
- Serves as the core of GitHub's collaborative power.

43.3.1.5 Additional Tools

- **Wiki, Security, and Insights** – Used for advanced communication and project analysis.
- **Settings** – Allows customization, including renaming the repository and controlling access.

43.4 Summary

- You learned how to create and verify a GitHub account.
- Repositories store code, track issues, and enable collaboration.

- GitHub provides several tools like Issues, Pull Requests, and Projects to manage work efficiently.

44 Git and GitHub Basics

44.1 Creating a GitHub Account

Before proceeding, ensure you have registered for a GitHub account and logged in.

44.2 Creating a New Repository

To create a new repository, follow these steps:

1. Click + and select **New Repository**.
2. Provide the following details:
   - **Repository Name**: Enter a name for your repository. o **Description (Optional)**: Add a brief description of the repository.
   - **Visibility**: Choose whether the repository should be **public** or **private**.
   - **Initialize with a README**: Select this option to include a README file.
3. Click **Create Repository**.

44.3 Viewing Your Repository

Once created, you will be redirected to your repository's main page. The root folder contains a single file: **README.md**.

44.4 Editing the README File

To edit the README file directly in your browser:

1. Click the **pencil icon** to open the online editor.
2. Modify the text as needed.
3. Scroll down to the **Commit changes** section.
4. Enter a commit message and optionally add a description.
5. Click **Commit changes** to save your edits.

44.5 Creating a New File

GitHub provides a built-in web editor to create files:

1. Click **Add File** and select **Create New File**.
2. Enter a **file name** (e.g., firstpython.py).
3. Add a comment describing the file's purpose.
4. Write the code for the file.
5. Click **Commit changes** to save the new file. **44.6 Editing an Existing File**

To edit a file:

1. Click the file name to open it.
2. Click the **pencil icon** to enter the editing mode.
3. Make the necessary changes.
4. Click **Commit changes** to save your edits. **44.7**

Uploading a File from Your Local System

To upload a file:

1. From the repository home screen, click **Add File** and choose **Upload Files**.
2. Click **Choose Your Files** and select the files to upload.
3. Once the upload completes, click **Commit Changes**.
4. The repository now reflects the uploaded files.

44.8 Summary

In this tutorial, you learned how to:

- Create a new repository on GitHub.
- Edit and commit changes to files using the web interface.
- Create, modify, and upload files to a repository.

These fundamental skills are essential for managing projects on GitHub efficiently.

45 GitHub working with Branches

In the lecture "GitHub: Working with Branches," you learn about the concept of **branches** in GitHub, which are snapshots of your repository allowing you to make changes without affecting the main codebase (master branch). Key points include:

- **Branch Creation**: You can create a child branch from the master branch to test and develop changes.

- **Merging**: Once changes in the child branch are satisfactory, you can merge them back into the master branch.
- **Pull Requests (PR)**: A PR is used to propose changes and notify team members for review before merging into the master branch. It shows differences between branches and facilitates collaboration.

This process helps maintain a stable master branch while allowing for experimentation and development in separate branches.

46 Hands-On Lab: Branching and Merging (Web UI)

After completing this lab, you will be able to:

1. Create a branch
2. Commit changes to a child branch
3. Open a pull request
4. Merge a pull request into the main branch

47 Getting Started with Branches using Git Commands

You would typically use Git commands from your own desktop/laptop. However, so you can get started using the commands quickly without having to download or install anything, we are providing an IDE with a Terminal on the Cloud. Simply click the Open Tool button below to launch the Skills Network Cloud IDE and in the new browser tab that launches, follow the instructions to practice the Git commands. After completing this lab you will be able to use git commands to start working with creating and managing your code branches, including:

1. create a new local repository using **git init**
2. create and add a file to the repo using **git add**
3. commit changes using **git commit**
4. create a branch using **git branch**
5. switch to a branch using **git checkout**
6. check the status of files changed using **git status**
7. review recent commits using **git log**
8. revert changes using **git revert**
9. get a list of branches and active branch using **git branch**
10. merge changes in your active branch into another branch using **git merge**

48 Module 5 Summary

- The capabilities of R and its uses in Data Science.
- The RStudio interface for running R codes.
- Popular R packages for Data Science.
- Popular data visualization packages in R.
- Plotting with the inbuilt R plot function.

  Plotting with ggplot.

-
- Adding titles and changing the axis names using the ggtitle and lab's function.
- A Distributed Version Control System (DVCS) keeps track of changes to code, regardless of where it is stored.
- Version control allows multiple users to work on the same codebase or repository, mirroring the codebase on their own computers if needed, while the distributed version control software helps manage synchronization amongst the various codebase mirrors.
- Repositories are storage structures that:
    - Store the code
    - Track issues and changes
    - Enable you to collaborate with others
- Git is one of the most popular distributed version control systems.
- GitHub, GitLab and Bitbucket are examples of hosted version control systems.
- Branches are used to isolate changes to code. When the changes are complete, they can be merged back into the main branch.
- Repositories can be cloned to make it possible to work locally, then sync changes back to the original.

Module 7

49 Introduction to WatsonX.AI

49.1 What is WatsonX.AI?

WatsonX.AI is a collaborative platform designed for the data science community. It is used by data analysts, data scientists, generative AI engineers, data engineers, developers, and data stewards to analyze data and construct models.

49.2 Features and Capabilities of WatsonX.AI

49.2.1 Project Management

- Create projects to organize data connections, assets, and Jupyter notebooks.
- Upload, clean, and shape data for analysis.
- Share data visualizations via dashboards without coding.

49.2.2 Model Building and AI Capabilities

- Build, train, and deploy machine learning models.
- Perform prompt engineering and tuning for large language models.
- Customize views for different tasks such as data science, data engineering, data curation, and AI.

49.3 Availability and Deployment

49.3.1 Cloud and On-Premises Options

- **IBM Cloud and AWS**: Available as a software-as-a-service offering.
  Cloud Pak for Data: Provides built-in integration with other data services.
- **OpenShift**: Can be deployed on any private or public cloud with OpenShift.

•

49.4 IBM Cloud Pak for Data

49.4.1 Overview

Cloud Pak for Data is a secure, seamless data access and integration platform. It enables a single view of data across multiple sources and is available:

- As a service on Cloud.
- On private and public clouds when installed on OpenShift.

49.4.2 Components of Cloud Pak for Data

- IBM WatsonX.AI
- IBM Watson Knowledge Catalog
- IBM Watson Machine Learning
- Other integrated data services

49.5 Navigating WatsonX.AI

49.5.1 Dashboard and Navigation

- **Projects**: Shows projects and jobs created.
- **Deployments**: Train, deploy, and manage machine learning models.
- **Projects Hub**: Collection of datasets, notebooks, industry accelerators, and sample projects.

49.5.2 Creating a Project

1. Click the **plus sign** under the Project section.
2. Assign a **project name**.
3. Assign an **object storage bucket** to store project artifacts.

49.5.3 Managing Project Assets

- **Overview Page**: Displays recent assets, resource usage, project history, and readme files.
- **Assets Tab**: Allows creation and management of datasets, models, and pipelines.
- **New Asset Button**: Enables creation of analytical assets like flows, visualizations, experiments, or notebooks.
- **Import Asset Button**: Allows importing external assets.

49.6 Tools and Services in WatsonX.AI

49.6.1 Development Environments

- **RStudio IDE**: Supports R notebooks and scripts.
- **Jupyter Notebooks**: Provides a programmatic approach to machine learning model training.

49.6.2 Job Execution and Management

- **Jobs Tab**: Execute and schedule jobs for running assets like data refinery flows and notebooks.
- **Manage Tab**: Control user access, define environments, monitor runtimes, and track resource usage.

49.6.3 Services and Integrations

- Associate IBM Cloud services with projects for added tools and capabilities.
- Integrate with third-party tools like Git repositories and JupyterLab.

49.7 Data Preparation and AI Model Tools

49.7.1 Data Preparation Tools

- **Connections Tool**: Connect to data sources.
- **Synthetic Data Generation**: Create synthetic datasets.
- **Data Refinery**: Prepare and visualize data.

49.7.2 AI and Model Development Tools

- **Generative AI Tools**: Prompt Lab and Tuning Studio.
- **Decision Optimization**: Solve optimization scenarios.
- **SPSS Modeler**: Develop predictive models for business operations.
- **Studio AI**: Wizard-based tool for automatic training of machine learning models.

49.7.3 Automating Model Lifecycles

- **Pipeline Management Tool**: Automates the lifecycle of AI models.

49.8 Summary

- WatsonX.AI enables data analysis, model building, and AI development.
- Available on IBM Cloud, AWS, and OpenShift.
- Integrates with Cloud Pak for Data for seamless data access and management.
- Provides a variety of tools for data preparation, AI modeling, and automation.
- Learning WatsonX.AI enhances career opportunities with accessible and easyto-use resources.

50 Creating an IBM Cloud Account and a Watson X Service

50.1 Creating an IBM Cloud Account

To create an IBM Cloud Account, follow these steps:

1. **Go to the IBM Cloud Registration page.**
2. **Enter your email and password**, then click **Next**.
3. **Verify your email** using the 7-digit code sent to your email, then click **Next**.
4. **Enter your details**: o First Name o Last Name o Country or Region o Click **Next**.
5. **Review the Account Notice** and opt-in for email updates if desired.
6. **Accept the terms and conditions**, then click **Continue**.
7. **Review and accept the Privacy Notice**, then click **Continue**.
8. **Apply Feature Code** (automatically applied) and click **Create Account**.
9. **Wait for account setup completion.**

Once your IBM Cloud account is created, you will see the **IBM Cloud Dashboard**.

50.2 Creating an IBM Watson X Service

To use the Watson Studio Service:

1. Click **Catalog** on the IBM Cloud Dashboard.
2. Scroll down and select **AI/Machine Learning**.
3. Select **Watson Studio**, which will open the Watson Studio creation page.
4. Choose a **Location** (Dallas or London).
5. To avoid charges, select the **Lite plan**.
6. Accept the license agreements and click **Create**.
7. Click **Launch in IBM Watson X**.
8. Provide a **Company Name and Phone Number**, then select your preferred contact options.
9. Click **Continue**.
10. On the next screen, click **I Agree** to the Terms.
11. Click the **Close icon**.

You have successfully created an IBM Watson Studio account and are now ready to use the Watson X service.

50.3 Creating a Project in Watson X

To create a new project:

1. Click **Create a new project**.
2. Provide a **Project Name and Description**.
3. The **Cloud Object Storage** will be automatically added.
4. Click **Create**.

5. Wait a few seconds for the project setup to complete.

Once the project is created, you will see the project dashboard.

50.4 Summary

- An **IBM Cloud Account** is required to use Watson X.
- You can access **Watson Studio** by navigating to **Catalog > AI/Machine Learning**.
- Projects can be created from scratch, from samples, or from files using the **Create a new project** option.

The next video will guide you through adding a notebook to your project. Now, let's begin exploring Watson X!

51 Jupyter Notebooks in Watson Studio – Part 1

51.1 Creating a Jupyter Notebook

To create a new Jupyter Notebook in Watson Studio:

1. Navigate to the project home page.
2. Click **New asset** to add or create a new notebook.
3. Under **Tool type**, select **Code editors** and then choose **Jupyter notebook editor**.
4. On the **New Notebook** page, you can:
    - Create a blank notebook. o Upload a notebook file from your file system.
    - Upload a notebook file from a URL.
5. Provide a notebook name and description.

51.1.1 Specifying the Runtime Environment

- Choose the runtime environment for the desired programming language (Python, R, or Scala).
- Click **Create** to generate the notebook.

51.2 Uploading Data to the Notebook

1. Click **Find and Add Data**.
2. In the **Data** pane, browse for the files or drag them onto the pane.
3. Stay on the page until the upload is complete.
4. Click **Insert to code** and select **pandas DataFrame**.
5. Insert a cell at the top of the notebook using **Insert Cell Above** from the **Insert** tab.
6. Change the cell type to **Markdown** and describe the notebook.
7. Run the Markdown cell and proceed to execute the notebook.

51.3 Running and Saving the Notebook

- The inserted code loads the dataset into a DataFrame.
- Run the code cell to display the first five rows of the dataset.
- From the **File** tab, select **Save Version** to save the latest changes.
- Click the project name to return to the project home page.

51.4 Accessing and Editing the Notebook

1. On the project home page, under the **Assets** tab, select the **Source Code** tab.
2. Click to open the recently used notebook (it will open in **read-only mode**).
3. Click the **pencil icon** in the **Notebook action bar** to enable editing.

51.4.1 Viewing Notebook Information

- Click the **View Notebook Info** icon.
- Under the **General** tab, you can:
  - Change the name and description.
  - View the last editor, last modified date, and creation date.
- Under the **Environment** tab, you can: o View and change the environment template. o Check the runtime status.

51.5 Sharing a Jupyter Notebook

- Click the **Share** icon in the **Notebook action bar**.
- To share a read-only version:
  1. Enable **Share with anyone who has a link**.
  2. Choose sharing options:
     - **Only text and output**.
     - **All content excluding sensitive code cells**.
  3. Share via a link or social media.

51.6 Creating and Scheduling Jobs

1. Click the **Create a job** icon in the **Notebook action bar**.
2. Click **Create a job** and follow these steps:
   - **Define details**: Enter the job name and description, then click **Next**. o **Configure**: Fill in the required details and click **Next**.
   - **Schedule**:
     - Select a start day and time.
     - Choose whether to **repeat** the job or run it once. ▪ Click **Next**.
   - **Notifications**: Enable notifications if required, then click **Next**. o **Review** the job details and click **Create**.

51.6.1 Managing Jobs

- View the created jobs under the **Jobs** tab.
- From here, you can **edit** or **delete** jobs.

51.7 Summary

- A Jupyter Notebook can be created from the **New asset** option in the **Assets** tab.
- Data can be uploaded and processed within the notebook.
- Notebooks can be shared while protecting sensitive cells.
- Jobs can be created and scheduled using the **Create a job** feature and managed under the **Jobs** tab.

52 Disclaimer: Insert data to code on Watson

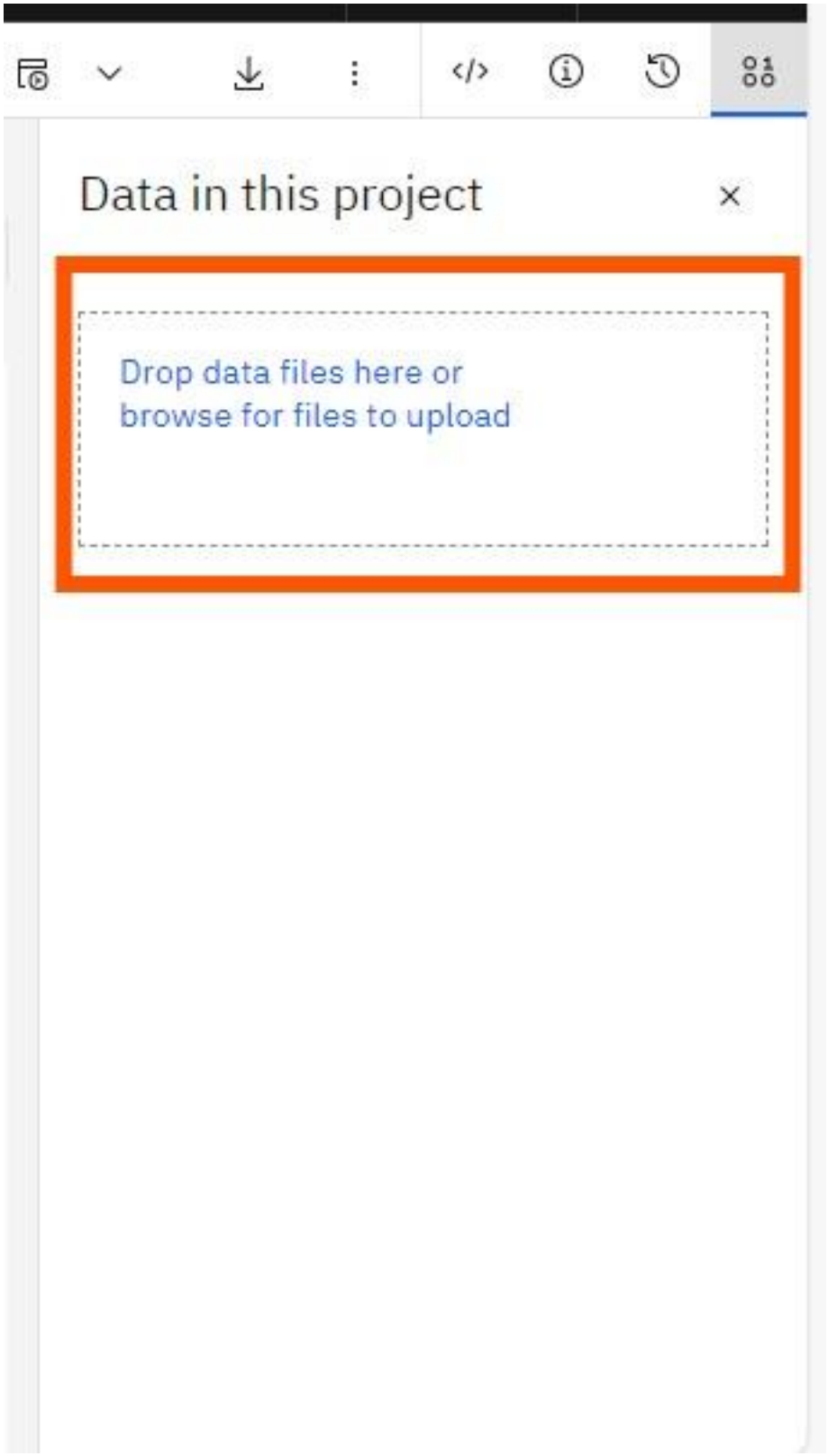
Disclaimer: Before proceeding, please note that the following instructions are specific to the new UI of Watson Studio. If you encounter any issues or need further assistance, feel free to ask for help.

Instructions:

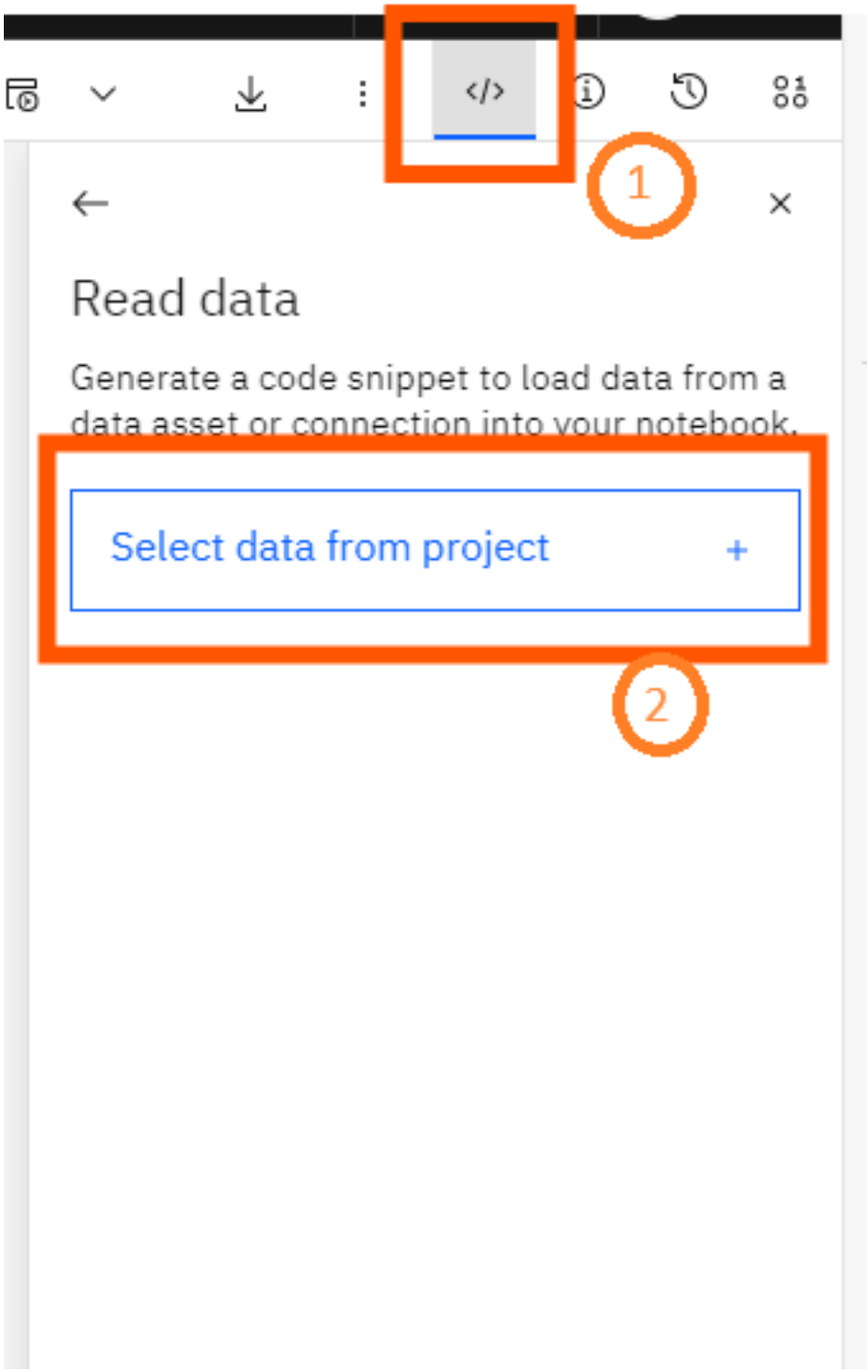
1. Access Watson Studio.
2. Follow the prompts to generate the code based on your requirements.

Here are the steps to be followed:

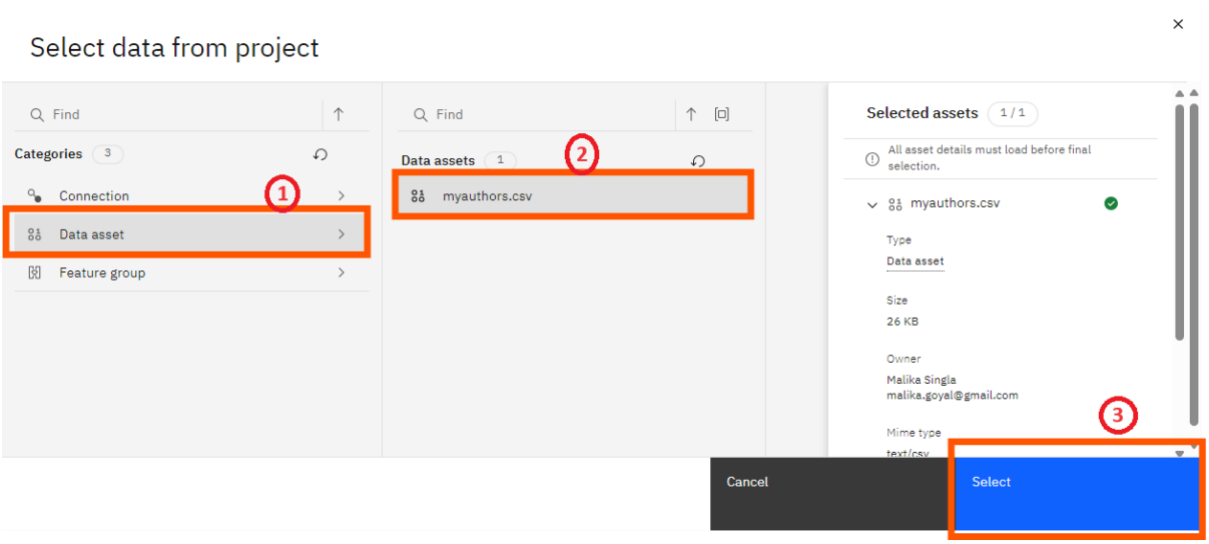
1. Upload your data file in CSV format or any other compatible format to the data pane.



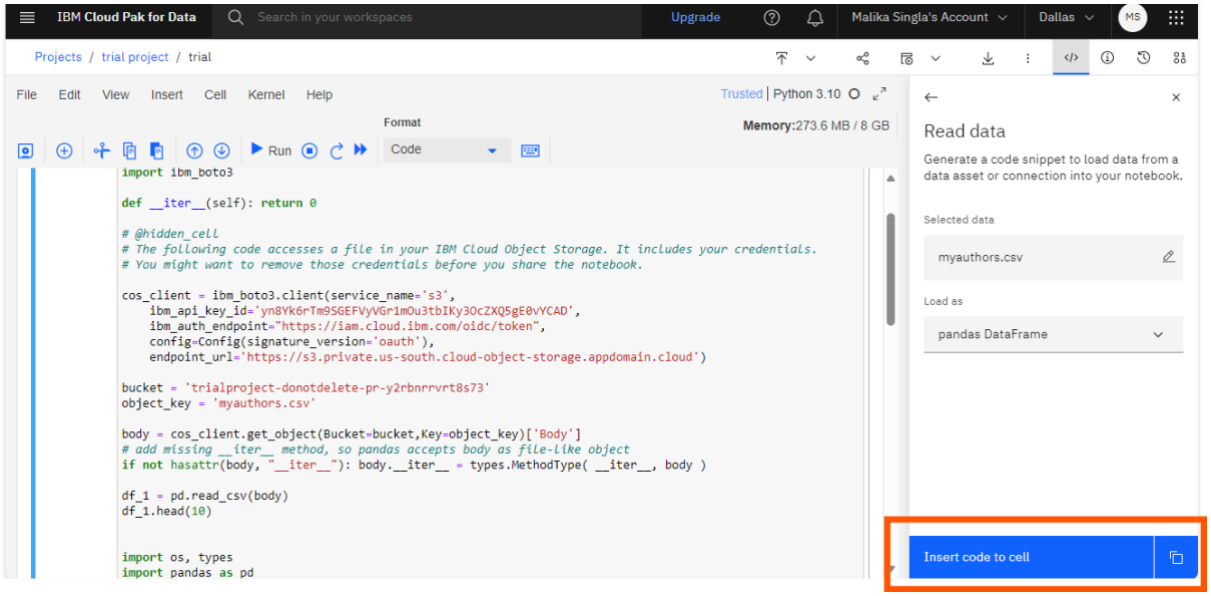
2. Once you have uploaded the data, tap on the **code (</>) icon**, from the left side of the data pane. and tap on the button "**select data from project"**.



3. Click on "**Data Assets**," then, select the uploaded data, and tap on "**Select.**"



4. Click on "**insert code to cell**" button to insert the code.



53 Jupyter Notebooks in WatsonX Part 2

53.1 Checking the Jupyter Notebook Runtime Environment

To manage the Jupyter Notebook runtime environment:

1. Go to the **Data Science Project** homepage.
2. Click the **Manage** tab.
3. Select **Environments** from the left pane.

If your notebook is executing, you will see the active runtime environment. If it is not visible, execute the notebook to check the active runtime.

53.2 Using Different Types of Jupyter Notebook Templates

To explore available runtime environment templates:

1. Click **Templates** in the **Environments** tab.
2. In the **Template Name** list, browse the available runtime environment templates.
3. Select a template to set it as the active runtime environment.
4. Upon selection, a summary of the template will be displayed.

53.2.1 Creating a New Template

1. Navigate back to the **Template** page.
2. Click **New Template**.
3. Enter the **environment name**, **description**, and **configuration**.
4. Click **Create** to finalize the new environment.
5. The newly created environment will appear at the top, displaying its summary and software configuration details.

53.3 Changing the Kernel Within a Jupyter Notebook

To switch to a newly created template runtime:

1. Click the **Assets** tab.
2. Select the new notebook you created.
3. If a **lock icon** appears, unlock it and select the notebook again.
4. Click the **three dots** on the right and select **Change Environment**.
5. Choose the newly created template environment and click **Change**.
6. In the pop-up, select the **kernel** from the drop-down menu and click **Select Kernel**.
7. The new runtime environment will be visible in the **top-right corner**.

53.4 Executing Code in the New Runtime Environment

- Drag and drop a **CSV file** to upload it.
- Click **Insert Code to Cell** to include the code in the notebook.
- Click **Run** to execute the notebook.
- The output will now reflect the new runtime environment.

53.5 Summary

- Jupyter Notebook runtime environments and templates are available in the **Environments** tab on the homepage.
- Users can create and change Jupyter Notebook templates.
- The runtime environment can be switched by selecting a new template and changing the kernel.

54 Linking GitHub to watsonx

54.1 Connecting a watsonx Account with GitHub

To integrate your watsonx account with GitHub, follow these steps:

1. Open your project and locate the data science notebook you created earlier.
2. Click **Manage**.
3. In the left panel, under **Project**, click **Service and Integrations**.
4. Under **IBM Services**, select **Third-party integrations**.
5. Click **Connect Integration**.
6. In **Connect Integration**, select **GitHub** and click **Next**.
7. Click the **personal access token** hyperlink to create a new access token.
8. Log in to your GitHub account by entering your username and password, then click **Sign in**.
9. To create a new personal access token:
    - Provide a description for the token. o Under **Select scopes**, select **repo**.
    - Ensure **gist** is selected and click **Generate Token**.
10. On the **Personal access tokens** page, copy the generated token by clicking the copy icon.
11. Return to the **Connect Integration** area and add the generated token.

54.2 Publishing Watsonx Notebooks on GitHub

Once your watsonx account is connected to GitHub, follow these steps to publish a notebook:

1. Locate the repository URL where you want to push the notebook:
    - Ensure you are in the **master** branch of the repository.
    - Click **Code** and copy the URL using the copy icon.
2. Return to the **Connect Integration** area and enter the copied repository URL.
3. Click **Connect**.
4. Once the repository is successfully added, a confirmation message will appear.
5. Navigate to the **Assets** tab and select the notebook you want to publish.
6. Open the notebook and, on the top right, select the down arrow.
7. Click **Publish on GitHub**.
8. In the **Publish on GitHub** page:
    - Select **All content except hidden code cells**.
    - Click **Publish**.
9. A success message will confirm that the notebook has been published with its name and repository location.
10. In GitHub, you will find the notebook file **datasciencenotebook.ipynb**, confirming the successful publication.

54.3 Summary

- You can integrate a watsonx account with GitHub.
- A personal access token is required to establish the connection.
- Notebooks from watsonx can be published to GitHub using the **Publish on GitHub** feature.

55 Module 6 Summary

Watson Studio is a helpful tool for:

- Analyzing and viewing data
- Cleaning and shaping data
- Embedding data into streams
- Creating, training, and deploying
- Learning Watson Studio promotes career growth.
- Learning Watson Studio is easy and requires no special skills.
- Watson Studio offers many available resources.
- You must create an IBM Cloud account to use Watson Studio.
- You can open Watson Studio by clicking Catalog on the IBM Cloud dashboard and then AI/Machine Learning.
- You can create an empty project or a project from a sample or file by selecting the Work with data option.
- You can add a storage service by clicking Add and then selecting the storage service of your choice.
- You can add or create a new notebook by clicking New asset on the project home page under the Assets tab.
- You can share your notebook without sharing the sensitive cells.
- Jobs are created and scheduled from the Create a job icon in the notebook action bar.
- Jobs can be viewed, edited, or deleted on the project home page under the Jobs tab.
- Jupyter Notebook runtime environments and templates can be found on the Environments tab in the Home Page.
- You can create and change new Jupyter Notebook templates.
- Watson Studio accounts can be integrated into GitHub.
- You can push notebooks from Watson Studio to GitHub.

55. Glossary

Term	Definition
Apache MLlib	Language that makes machine learning scalable.
Apache Spark	A general-purpose cluster-computing framework allowing you to process data using compute clusters.
API	Application Programming Interface; allows communication between two pieces of software.
Caffe	A deep learning algorithm repository built with C++ with Python and Matlab bindings.
CDLA	Community Data License Agreement.
Classification Models	Used to predict whether information or data belongs to a category (or "class").
CLI	Command Line Interface.
C++	A general-purpose programming language; an extension of the C programming language ("C with Classes").
Data Set	A structured collection of data.
Deeplearning4	Language for deep learning.
Deep Learning	A specialized type of machine learning that uses models and techniques inspired by the human brain to solve complex problems.
ELT	Extract, Load, Transform.
ETL	Extract, Transform, Load.
FSF	Free Software Foundation.
ggplot2	A popular library for data visualization in R.
GPU	Graphics Processing Unit.
Git	The de facto standard for code asset management (version control), with services such as GitHub and GitLab.
Hadoop	A Java-based framework that manages data processing and storage for big data applications running on clustered systems.
Java	An object-oriented programming language.
Java-ML	Language for machine learning.
JVM	Java Virtual Machine.
JavaScript	A general-purpose programming language that expanded beyond the browser with Node.js and other server-side technologies.
Julia	A language for high-performance numerical analysis and computational science.
Jupyter Notebook	A browser-based application for creating and sharing documents containing code, equations, visualizations, narrative text, and more.

JupyterLab	A browser-based application for accessing multiple Jupyter Notebooks, code files, and data files.
Kernel	An execution environment for different programming languages.
Lattice	A high-level data visualization library capable of creating graphics without extensive customization.
Library	A collection of functions and methods that perform tasks without requiring users to write all the code.
Leaflet	A library for creating interactive plots.
ML	Machine Learning; uses algorithms (models) to identify patterns in data.
Matplotlib	A package for data visualization.
Model Training	The process by which a model learns patterns from data.
MNIST	Modified National Institute of Standards and Technology dataset.
MongoDB	A NoSQL database for big data management built with C++.
NLP	Natural Language Processing.
NLTK	Natural Language Toolkit.
NumPy	A library based on arrays and matrices that supports mathematical operations and numerical computing.
OSI	Open Source Initiative.
PaaS	Platform as a Service.
Pandas	A library providing data structures and tools for data cleaning, manipulation, and analysis.
Plotly	A library for web-based data visualizations that can be displayed or saved as HTML files.
PMML	Predictive Model Markup Language.
Python	A high-level, general-purpose programming language with a rich standard library for databases, automation, web scraping, text processing, image processing, machine learning, and data analytics.
R	A statistical computing language.
Regression Models	Used to predict numeric (real-valued) outputs.
Reinforcement Learning	A learning approach loosely based on how humans and other organisms learn through interaction.
REST	Representational State Transfer.
RStudio	An integrated development environment for programming, debugging, remote data access, data exploration, and visualization.

SaaS	Software as a Service.
Scala	A general-purpose programming language combining object-oriented and functional programming with a strong static type system.
Spyder	An IDE that integrates code, documentation, and visualizations into a single workspace.
SQL	Structured Query Language; a non-procedural language used for querying and managing data.
Supervised Learning	A machine learning approach where humans provide input data along with correct outputs.
TensorFlow	A deep learning library for dataflow built with C++.
Unsupervised Learning	A learning approach where data is unlabeled; examples include clustering models that group similar records.
Watson Studio	A fully integrated development environment for data scientists.
Weka	A language/tool for data mining.